



## Testing

*The software is done.*

*We are just trying to get it to work.*

*—Statement made in a Joint STARS E-8A FSD  
Executive Program Review*

**T**esting is the process of finding differences between the expected behavior specified by system models and the observed behavior of the system. Unit testing finds differences between the object design model and its corresponding component. Structural testing finds differences between the system design model and a subset of integrated subsystems. Functional testing finds differences between the use case model and the system. Finally, performance testing finds differences between nonfunctional requirements and actual system performance. When differences are found, developers identify the defect causing the observed failure and modify the system to correct it. In other cases, the system model is identified as the cause of the difference, and the model is updated to reflect the state of the system.

From a modeling point of view, testing is the attempt of falsification of the system with respect to the system models. The goal of testing is to design tests that exercise defects in the system and to reveal problems. This activity is contrary to all other activities we described in previous chapters: Analysis, design, implementation, communication and negotiation are constructive activities. Testing, however, is aimed at breaking the system. Consequently, testing is usually accomplished by developers that were not involved with the construction of the system.

In this chapter, we first motivate the importance of testing. We provide a bird's eye view of the testing activities, we describe in more detail the concepts of fault, error, failure, and test, and then we describe the testing activities that result in the plan, design, and execution of tests. We conclude this chapter by discussing management issues related to testing.

## 9.1 Introduction

Testing is the process of analyzing a system or system component to detect the differences between specified (required) and observed (existing) behavior. Unfortunately, it is impossible to completely test a nontrivial system. First, testing is not decidable. Second, testing needs to be performed under time and budget constraints. As a result, systems are deployed without being completely tested, leading to faults discovered by end users.

The first launch of the Space Shuttle Columbia in 1981, for example, was canceled because of a problem that was not detected during development. The problem was traced to a change made by a programmer 2 years earlier, who erroneously reset a delay factor from 50 to 80 milliseconds. This added a probability of 1/67 that any space shuttle launch would fail. Unfortunately, despite of thousands of hours of testing after the change was made, the fault was not discovered during the testing phase. During the actual launch the fault caused a synchronization problem with the shuttle's five on-board computers that then led to the decision to abort the launch. The following is an excerpt of an article by Richard Feynman that describes the challenges of testing the Space Shuttle.<sup>1</sup>

In a total of about 250,000 seconds of operation, the engines have failed seriously perhaps 16 times. Engineering pays close attention to these failings and tries to remedy them as quickly as possible. This it does by test studies on special rigs experimentally designed for the flaws in question, by careful inspection of the engine for suggestive clues (like cracks), and by considerable study and analysis. ...

The usual way that such engines are tested (for military or civilian aircraft) may be called the component system, or bottom-up test. First it is necessary to thoroughly understand the properties and limitations of the materials to be used (for turbine blades, for example), and tests are begun in experimental rigs to determine those. With this knowledge larger component parts (such as bearings) are designed and tested individually. As deficiencies and design errors are noted they are corrected and verified with further testing. Since one tests only components at a time, these tests and modifications are not overly expensive. Finally one works up to the final design of the entire engine, to the necessary specifications. There is a good chance, by this time that the engine will generally succeed, or that any failures are easily isolated and analyzed because the failure modes, limitations of materials, etc., are so well understood. There is a very good chance that the modifications to the engine to get around the final difficulties are not very hard to make, for most of the serious problems have already been discovered and dealt with in the earlier, less expensive, stages of the process.

The Space Shuttle Main Engine was handled in a different manner, top down, we might say. The engine was designed and put together all at once with relatively little detailed preliminary study of the material and components. Then when troubles are found in the bearings, turbine blades, coolant pipes, etc., it is more expensive and difficult to discover the causes and make changes. For example, cracks have been found in the turbine blades of the high pressure oxygen turbopump. Are they caused by flaws in the material, the effect of the oxygen atmosphere on the properties of the material, the thermal stresses of start-up or shutdown, the vibration and stresses of steady running, or mainly at some resonance at certain speeds, etc.? How long can we run from crack initiation to crack failure, and how does this depend on power level? Using the completed engine as a test bed to resolve such questions is extremely expensive. One does not wish to lose an entire engine in order to find out where and how failure occurs.

Yet, an accurate knowledge of this information is essential to acquire a confidence in the engine reliability in use. Without detailed understanding, confidence can not be attained. A further disadvantage of the top-down method is that, if an understanding of a fault is obtained, a simple fix, such as a new shape for the turbine housing, may be impossible to implement without a redesign of the entire engine.

The Space Shuttle Main Engine is a very remarkable machine. It has a greater ratio of thrust to weight than any previous engine. It is built at the edge of, or outside of, previous engineering experience. Therefore, as expected, many different kinds of flaws and difficulties have turned up. Because, unfortunately, it was built in the top-down manner, they are difficult to find and fix. The design aim of a lifetime of 55 missions equivalent firings (27,000 seconds of operation, either in a mission of 500 seconds, or on a test stand) has not been obtained. The engine now requires very frequent maintenance and replacement of important parts, such as turbopumps, bearings, sheet metal housings, etc. The high-pressure fuel turbopump had to be replaced every three or four mission equivalents (although that may have been fixed, now) and the high pressure oxygen turbopump every five or six. This is at most ten percent of the original specification.

Feynman's article gives us an idea of the problems associated with testing complex systems. Even though the space shuttle is an extremely complex hardware and software system, the testing challenges are the same for any complex system.

Testing has often been viewed as a job that can be done by beginners. Managers would assign the new members to the testing team, because the experienced people detested testing or were needed for the more important jobs of analysis and design. Unfortunately, such an attitude leads to many problems. To test a system effectively, a tester must have a detailed understanding of the whole system, ranging from the requirements to system design decisions and implementation issues. A tester must also be knowledgeable of testing techniques and apply these techniques effectively and efficiently to meet time, budget, and quality constraints.

This chapter is organized as follows. In Section 9.2, we take a bird's eye view of testing and show its relationship with other quality assurance methods. In Section 9.3, we define in more detail the model elements related to testing, including faults, their manifestation, and their relationship to testing. In Section 9.4, we describe the testing activities found in the development process. We first describe unit testing, which focuses on finding faults in a single component. We then describe integration and system testing, which focus on finding faults in combination of components and in the complete system, respectively. We also describe testing activities that focus on nonfunctional requirements, such as performance tests and stress tests. We conclude

1. Feynman [Feynman, 1988] wrote the article while he was a member of the Presidential commission investigating the explosion of the space shuttle Challenger in January 1985. The cause of the accident was traced to an erosion of the O-rings in the solid booster rockets. In addition to the testing problems of the main shuttle engine and the solid rocket boosters, the article mentions the phenomenon of gradually changing critical testing acceptance criteria and problems resulting from miscommunication between management and developers typically found in hierarchical organizations.

Section 9.4 with the activities of field testing and installation testing. In Section 9.5, we discuss management issues related to testing.

## 9.2 An overview of testing

**Reliability** is a measure of success with which the observed behavior of a system conforms to the specification of its behavior. **Software reliability** is the probability that a software system will not cause the failure of the system for a specified time under specified conditions [IEEE Std. 982-1989]. **Failure** is any deviation of the observed behavior from the specified behavior. An **error** means the system is in a state such that further processing by the system will lead to a failure, which causes the system to deviate from its intended behavior. A **fault**, also called defect or bug, is the mechanical or algorithmic cause of an error. The goal of testing is to maximize the number of discovered faults, which then allows developers to correct them and increase the reliability of the system.

We define testing as the systematic attempt to *find errors in a planned way* in the implemented software. Contrast this definition with another commonly used definition that says that “testing is the process of demonstrating that *errors are not present*.” The distinction between these two definitions is important. Our definition does not mean we simply demonstrate that the program does what it is intended to do. The explicit goal of testing is to demonstrate the presence of faults. Our definition implies that you, the developer, are willing to dismantle things. Most activities of the development process are constructive: During analysis, design, and implementation, objects and relationships are identified, refined, and mapped onto a computer environment.

Testing requires a different thinking, in that developers try to detect faults in the system, that is, differences between the reality and the requirements. Many developers find it difficult to do. One reason is the way we use the word “success” during testing. Many project managers call a test case “successful” if it does not find an error; that is, they use the second definition of testing during development. However, because “successful” denotes an achievement, and “unsuccessful” means something undesirable, these words should not be used in this fashion during testing.

In this chapter, we treat testing as an activity based on the falsification of system models, which is based on Popper’s falsification of scientific theories [Popper, 1992]. According to Popper, when testing a scientific hypothesis, the goal is to design experiments that falsify the underlying theory. If the experiments are unable to break the theory, our confidence in the theory is strengthened and the theory is adopted (until it is eventually falsified). Similarly, in software testing, the goal is to identify errors in the software system (“falsify the theory”). If none of the tests have been able to falsify the software system behavior with respect to the requirements, it is ready for delivery.

In other words, a software system is released when the falsification attempts (“tests”) have increased the reliability of the software, ensuring a larger confidence that the software system does what it is supposed to do.

### 9.2.1 Quality control techniques

There are many techniques for increasing the reliability of a software system. Figure 9-1 focuses on three classes of techniques for avoiding faults, detecting faults, and tolerating faults. *Fault avoidance techniques* try to detect errors statically, that is, without relying on the execution of any of the system models, in particular the code model. *Fault detection techniques*, such as debugging and testing, are uncontrolled and controlled experiments, respectively, used during the development process to identify errors and find the underlying faults before releasing the system. *Fault tolerance techniques* assume that a system can be released with errors and that system failures can be dealt with by recovering from them at run time.

### 9.2.2 Fault avoidance techniques

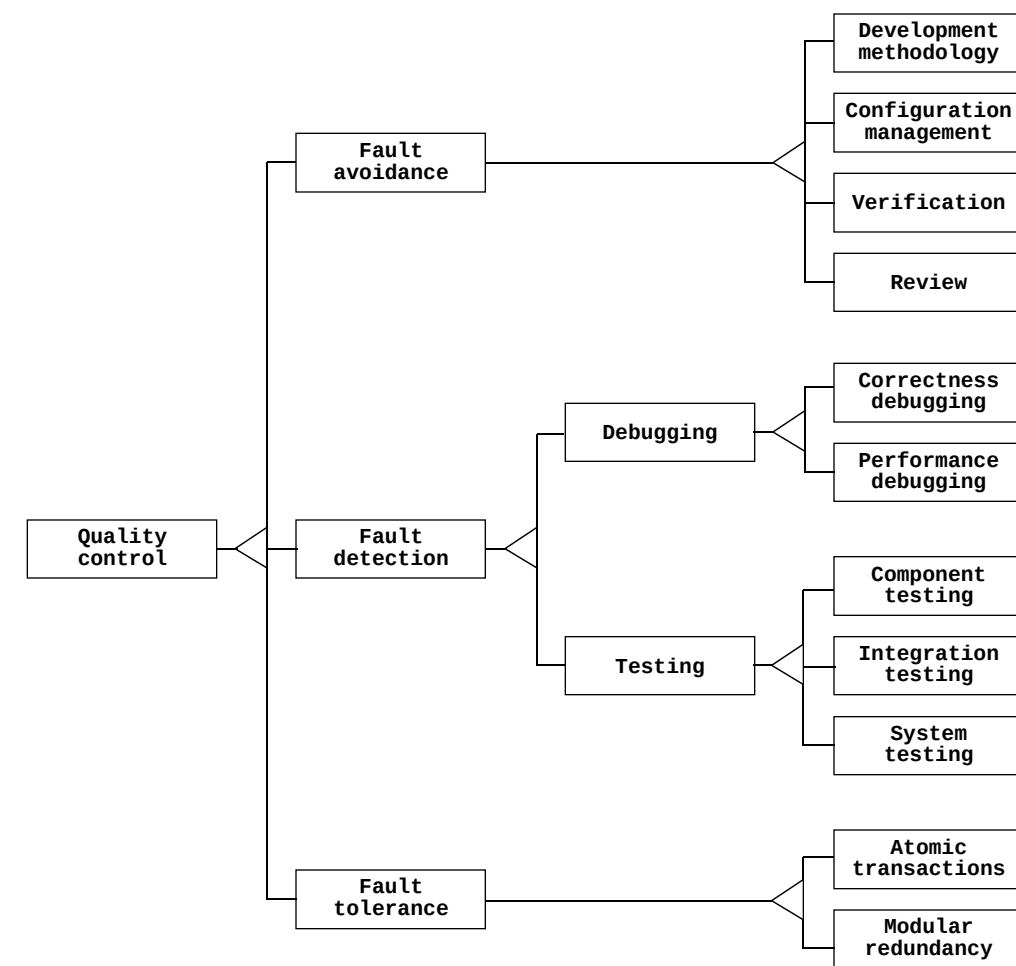
Fault avoidance tries to prevent the occurrence of errors and failures by finding faults in the system before it is released. Fault avoidance techniques include:

- development methodologies
- configuration management
- verification techniques
- reviews

**Development methodologies** avoid faults by providing techniques that minimize fault introduction in the system models and code. Such techniques include the unambiguous representation of requirements, the use of data abstraction and data encapsulation, minimizing of the coupling between subsystems and maximizing of subsystem coherence, the early definition of subsystem interfaces, and the capture of rationale information for maintenance activities. We described these techniques in Chapters 4–7. The assumptions of most of these techniques is that the system contains many less faults if complexity is dealt with model-based approaches.

**Configuration management** avoids faults caused by undisciplined change in the system models. For example, it is a common mistake to change a subsystem interface without notifying all developers of calling components. Configuration management can make sure that, if analysis models and code are becoming inconsistent with one another, analysts and implementor are notified. The assumption behind configuration management is that the system contains many less faults if change is controlled.

**Verification** attempts to find faults before any execution of the system. Verification is possible in specific cases, such as the verification of a small operating system kernel [Walker et al., 1980]. Verification, however, has limits. It is not in a mature enough state that it can be applied to assure the quality of large complex systems. Moreover, it assumes that the requirements are correct, which is seldom the case: Assume we need to verify an operation and we know the pre- and postconditions for the operation. If we can verify that the operation provides a transformation such that, before the execution of the operation the precondition is



**Figure 9-1** Taxonomy of quality control activities (UML class diagram).

true, and after the execution the postcondition holds, then the operation is correctly implemented; that is, it is verified. However, sometimes we forget important parts of the pre- or postcondition. For example, the postconditions might not state that the train should stay on the ground while moving from track 1 to track 2. Finally, verification addresses only algorithmic faults: It cannot deal with mechanical faults in the virtual machine. For verification, we must assume that the virtual machine on which the system is executing is correct and does not change during the proof.

A **review** is the manual inspection of parts or all aspects of the system without actually executing the system. There are two types of reviews: walkthrough and inspection. In a code

**walkthrough**, the developer informally presents the API, the code, and associated documentation of the component to the review team. The review team makes comments on the mapping of the analysis and object design to the code using use cases and scenarios from the analysis phase. An **inspection** is similar to a walkthrough, but the presentation of the unit is formal. In fact, in a code inspection, the developer is not allowed to present the artifacts (models, code, and documentation). This is done by the review team, which is responsible for checking the interface and code of the component against the requirements. It also checks the algorithms for efficiency with respect to the nonfunctional requirements. Finally, it checks comments about the code and compares them with the code itself to find inaccurate and incomplete comments. The developer is only present in case the review needs clarifications about the definition and use of data structures or algorithms.

Code reviews have proven to be very effective at detecting errors. In some experiments, up to 85% of all identified faults have been found in reviews [Fagan, 1976], [Jones, 1977].

### 9.2.3 Fault detection techniques

Fault detection techniques assist in finding faults in systems but do not try to recover from the failures caused by them. In general, fault detection techniques are applied during development, but in some cases they are also used after the release of the system. The black boxes in airplanes logging the last few minutes of a flight before the crash is an example of a fault detection technique. We distinguish two types of fault detection techniques, debugging and testing.

**Debugging** assumes that faults can be found by starting from an unplanned failure. The developer moves the system through a succession of states, ultimately arriving at and identifying the erroneous state. Once this state has been identified, the algorithmic or mechanical fault causing this state needs to be determined. There are two types of debugging: The goal of **correctness debugging** is to find any deviation between the observed and specified functional requirements. **Performance debugging** addresses the deviation between observed and specified nonfunctional requirements, such as response time.

**Testing** is a fault detection technique that tries to create failures or errors in a planned way. This allows the developer to detect failures in the system before it is released to the customer. Note that this definition of testing implies that a successful test is a test that identifies errors. We will use this definition throughout the development phases. Another often-used definition of testing is that “it demonstrates that errors are not present.” We will use this definition only after the development of the system when we try to demonstrate that the delivered system fulfills the functional and nonfunctional requirements.

If we would use this second definition all the time, we would tend to select test data that have a low probability of causing the program to fail. If, on the other hand, the goal is to demonstrate that a program has errors, we tend to look for test data with a higher probability of finding errors. The characteristics of a good test model is that it contains test cases that identify errors. Every input that can possibly be given to the system should be tested; otherwise, there is

a chance that faults are not detected. Unfortunately, such an approach requires extremely large testing times for even small systems.

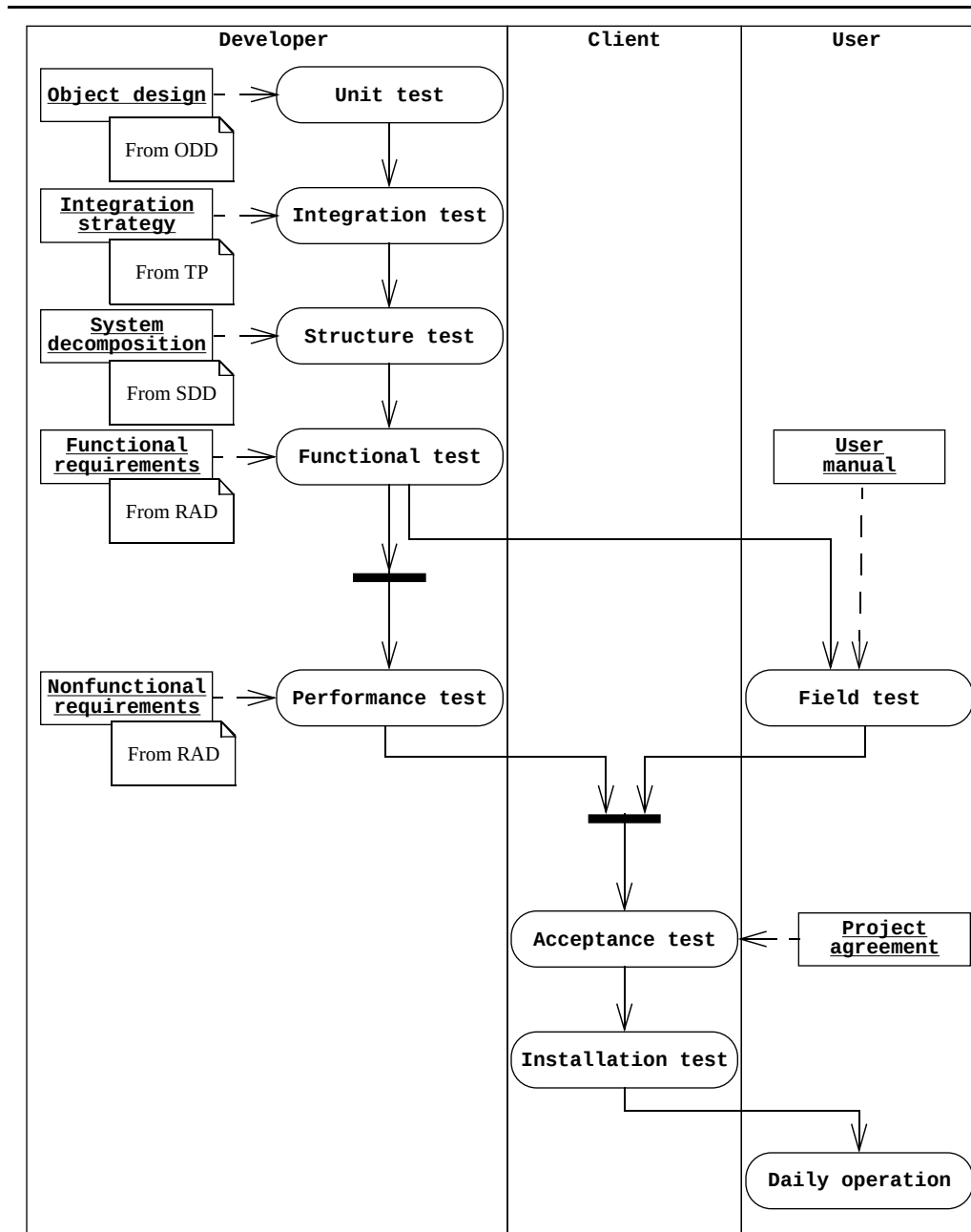
Figure 9-2 depicts an overview of the testing activities:

- **Unit testing** tries to find faults in participating objects and/or subsystems with respect to the use cases from the use case model
- **Integration testing** is the activity of finding faults when testing the individually tested components together, for example, subsystems described in the subsystem decomposition, while executing the use cases and scenarios from the RAD.
  - The **system structure testing** is the culmination of integration testing involving all components of the system. Integration tests and structure tests exploit knowledge from the SDD using an integration strategy described in the Test Plan (TP).
- **System testing** tests all the components together, seen as a single system to identify errors with respect to the scenarios from the problem statement and the requirements and design goals identified in the analysis and system design, respectively:
  - **Functional testing** tests the requirements from the RAD and, if available, from the user manual.
  - **Performance testing** checks the nonfunctional requirements and additional design goals from the SDD. Note that functional and performance testing are both done by developers.
  - **Acceptance testing** and **installation testing** check the requirements against the project agreement and should be done by the client, if necessary with support by the developers.

#### 9.2.4 Fault tolerance techniques

If we cannot prevent errors we must accept the fact that the released system contains faults that can lead to failures. Fault tolerance is the recovery from failure while the system is executing. It allows a system to recover from a failure of a component by passing the erroneous state information back to the calling components, assuming one of them knows what to do in this case. Database systems provide atomic transactions to recover from failure during a sequence of actions that need to be executed together or not at all. Modular redundancy is based on the assumption that system failures are usually based on component failures. Modular redundant systems are built by assigning more than one component to perform the same operation. The five onboard computers in the space shuttle are an example of a modular redundant system. The system can continue even if one component is failing, because the other components are still performing the required functionality. The treatment of fault tolerant techniques is important for highly reliable systems but is beyond the scope of this book. An excellent coverage of the topic is provided in [Siewiorek & Swarz, 1992].





**Figure 9-2** Testing activities and their related work products (UML activity diagram). Swimlanes indicate who executes the test.

### 9.3 Testing concepts

In this section, we present the model elements used during testing (Figure 9-3):

- A **component** is a part of the system that can be isolated for testing. A component can be an object, a group of objects, or one or more subsystems.
- A **fault**, also called bug or defect, is a design or coding mistake that may cause abnormal component behavior.
- An **error** is a manifestation of a fault during the execution of the system.
- A **failure** is a deviation between the specification of a component and its behavior. A failure is triggered by one or more errors.
- A **test case** is a set of inputs and expected results that exercises a component with the purpose of causing failures and detecting faults.
- A **test stub** is a partial implementation of components on which the tested component depends. A **test driver** is a partial implementation of a component that depends on the tested component. Test stubs and drivers enable components to be isolated from the rest of the system for testing.
- A **correction** is a change to a component. The purpose of a correction is to repair a fault. Note that a correction can introduce new faults [Brooks, 1975].

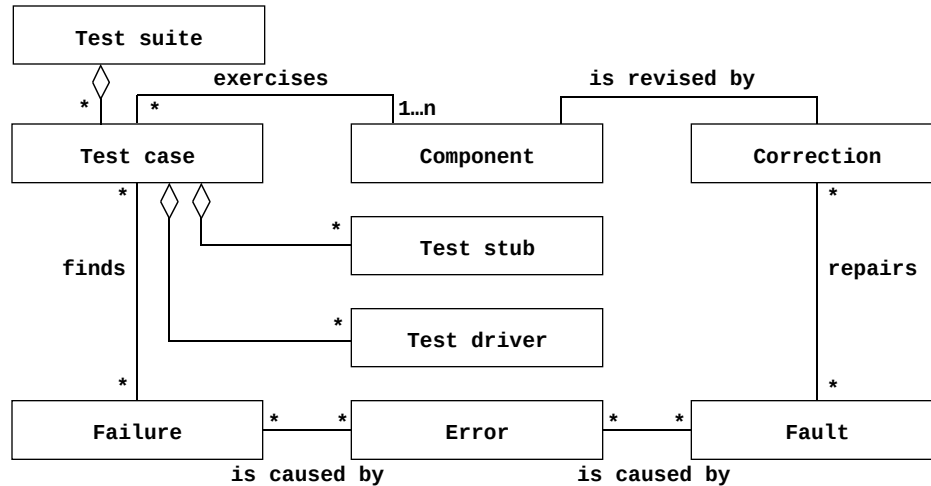
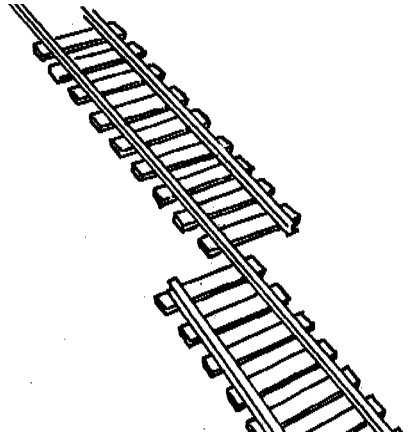


Figure 9-3 Model elements used during testing (UML class diagram).

### 9.3.1 Faults, errors, and failures

With the initial understanding of the terms from the definitions in Section 9.3, let's take a look at Figure 9-4.



**Figure 9-4** An example of a fault.

What do you see? Figure 9-4 shows a pair of tracks that are not aligned with each other. If we envision a train running over the tracks it would crash (fail). However, the figure actually does not present a failure nor an error nor a fault. It does not show a failure, because the expected behavior has not been specified nor is there any observed behavior. Figure 9-4 also does not show an error, because that would mean that the system is in a state that further processing will lead to a failure. We only see tracks here; no moving train is shown. To speak about error and failure, we need to compare the desired behavior (described in the use case in the RAD) with the observed behavior (described by the test case). Assume that we have a use case with a train moving from the upper left track to the lower right track (Figure 9-5).

|                             |                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <i>Use case name</i>        | DriveTrain                                                                                                                           |
| <i>Participating actor</i>  | TrainOperator                                                                                                                        |
| <i>Entry condition</i>      | Driver pushes the "StartTrain" button at the control panel.                                                                          |
| <i>Flow of events</i>       | <ol style="list-style-type: none"><li>1. The train starts moving on track 1.</li><li>2. The trains transitions to track 2.</li></ol> |
| <i>Exit condition</i>       | The train is running on track 2.                                                                                                     |
| <i>Special requirements</i> | none                                                                                                                                 |

**Figure 9-5** Use case DriveTrain specifying the expected behavior of the train.

|                                   |                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Test-case identifier</i>       | DriveTrain                                                                                                                                                                                                                                                                                                                                                    |
| <i>Test location</i>              | <a href="http://www12.in.tum.de/TrainSystem/test-cases/test1">http://www12.in.tum.de/TrainSystem/test-cases/test1</a>                                                                                                                                                                                                                                         |
| <i>Feature to be tested</i>       | Continuous operation of engine for 5 seconds                                                                                                                                                                                                                                                                                                                  |
| <i>Feature Pass/Fail Criteria</i> | The test passes if the train drives for 5 seconds and covers the length of at least two tracks.                                                                                                                                                                                                                                                               |
| <i>Means of control</i>           | 1. The StartTrain() method is called via a test driver StartTrain (contained in the same directory as the DriveTrain test).                                                                                                                                                                                                                                   |
| <i>Data</i>                       | 2. Direction of trip and duration are read from a input file<br><a href="http://www12.in.tum.de/TrainSystem/test-cases/input">http://www12.in.tum.de/TrainSystem/test-cases/input</a><br>3. If debug is set to TRUE, then the test case will output the system messages “Enter Track n, Exit Track n” for each n, where n is the number of the current track. |
| <i>Test Procedure</i>             | The test is started by double-clicking the test case at the specified location. The test will run without further intervention until completion. The test should take no more than 7 seconds.                                                                                                                                                                 |
| <i>Special requirements</i>       | The test stub Engine is needed for the track implementation.                                                                                                                                                                                                                                                                                                  |

**Figure 9-6** Test case DriveTrain for the use case described in Figure 9-5.

We can then proceed to derive a test case that moves the train from the state described in the entry condition of the use case to a state where it will crash, namely when it is leaving the upper track (Figure 9-6).

In other words, when executing this test case, we can demonstrate that the system contains a fault. Note, that the current state shown in Figure 9-7 is erroneous but does not show a failure.

The misalignment of the tracks can be a result of bad communication between the development teams (each track had to be positioned by one team) or because of a wrong implementation of the specification by one of the teams (Figure 9-8). Both of these are examples of algorithmic faults. You are probably already familiar with many other algorithmic faults that are introduced during the implementation phase. For example, “Exiting a loop too soon,” “exiting a loop too late,” “testing for the wrong condition,” “forgetting to initialize a variable” are all implementation-specific algorithmic faults. Algorithmic faults can also occur during analysis and system design. Stress and overload problems, for example, are design specific algorithmic faults that lead to failure when data structures are filled beyond their specified capacity. Throughput and performance errors are possible when a system does not perform at the speed specified by the nonfunctional requirements.

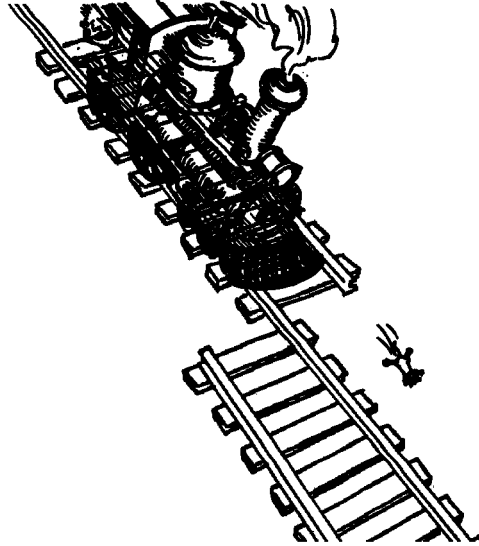


Figure 9-7 An example of an error.

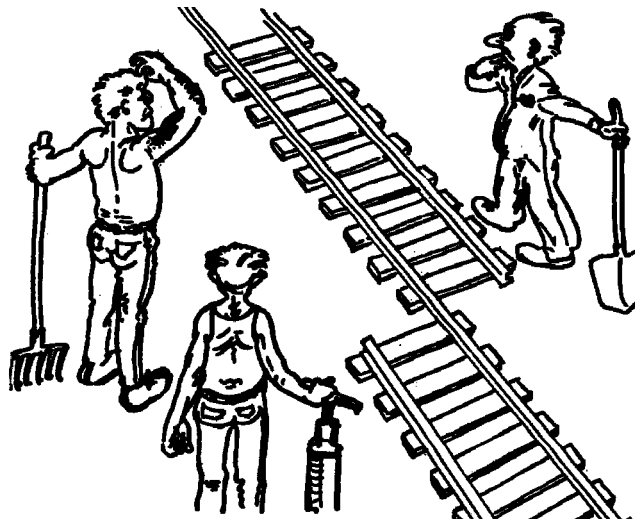
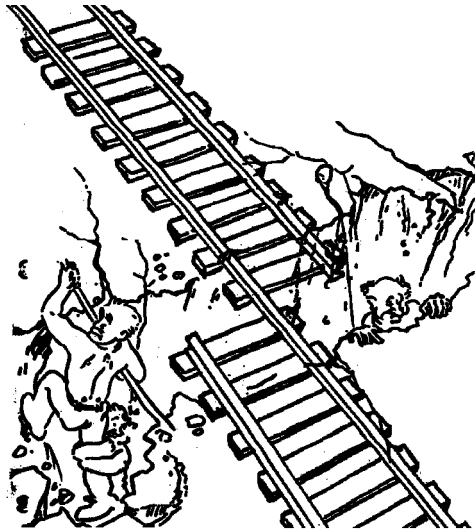


Figure 9-8 A fault can have an algorithmic cause.



**Figure 9-9** A fault can have a mechanical cause, such as an earthquake.

Even if the tracks are implemented according to the specification in the RAD, they could still end up misaligned during daily operation; for example, if an earthquake happens that moves the underlying soil (Figure 9-9).

An error in the virtual machine of a software system is another example of a mechanical fault: Even if the developers have done the correct implementation, that is, they have mapped the object model correctly onto the code, the observed behavior can still deviate from the specified behavior. In concurrent engineering projects, for example, where hardware is developed in parallel with software, we cannot always make the assumption that the virtual machine executes as specified. Other examples of mechanical faults are power failures. Note the relativity of the terms “fault” and “failure” with respect to a particular system component: The failure in one system component (the power system) is the mechanical fault that can lead to failure in another system component (the software system).

### 9.3.2 Test cases

A **test case** is a set of input data and expected results that exercises a component with the purpose of causing failures and detecting faults. A test case has five attributes: `name`, `location`, `input`, `oracle` and `log` (Table 9-1). The name of the test case allows the tester to distinguish between different test cases. A heuristic for naming test cases is to derive the name from the requirement it is testing or from the component being tested. For example, if you are testing a use case *Deposit()*, you might want to call the test case *Test\_Deposit*. If a test case involves two components A and B, a good name would be *Test\_AB*. The `location` attribute describes where

**Table 9-1** Attributes of the class TestCase.

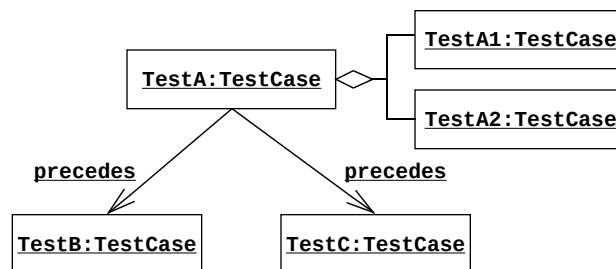
| Attributes | Description                                                            |
|------------|------------------------------------------------------------------------|
| name       | Name of test case                                                      |
| location   | Full path name of executable                                           |
| input      | Input data or commands                                                 |
| oracle     | Expected test results against which the output of the test is compared |
| log        | Output produced by the test                                            |

the test case can be found. It should be either the pathname or the URL to the executable of the test program and its inputs.

Input describes the set of input data or commands to be entered by the actor of the test case (which can be the tester or a test driver). The expected behavior of the test case is the sequence of output data or commands that a correct execution of the test should yield. The expected behavior is described by the `oracle` attribute. The `log` is a set of time-stamped correlations of the observed behavior with the expected behavior for various test runs.

Once test cases are identified and described, relationships among test cases are identified. Aggregation and the `preceeds` associations are used to describe the relationships between the test cases. Aggregation is used when a test case can be decomposed into a set of subtests. Two test cases are related via the `preceeds` association when one test case must precede another test case.

Figure 9-10 shows a test model where `TestA` must precede `TestB` and `TestC`. For example, `TestA` consists of `TestA1` and `TestA2`, meaning that once `TestA1` and `TestA2` are tested, `TestA` is tested; there is not separate test for `TestA`. A good test model has as few associations as possible, because tests that are not associated with each other can be executed independently from each other. This allows a tester to speed up testing, if the necessary testing



**Figure 9-10** Test model with test cases. `TestA` consists of two tests, `TestA1` and `TestA2`. `TestB` and `TestC` can be tested independently, but only after `TestA` has been performed.

resources are available. In Figure 9-10, TestBand TestC can be tested in parallel, because there is no relation between them.

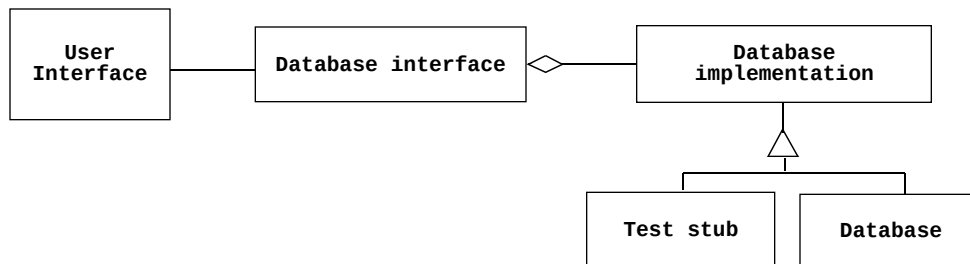
Test cases are classified into black box tests and white box tests, depending on which aspect of the system model is tested. **Black box tests** focus on the input/output behavior of the component. Black-box tests do not deal with the internal aspects of the component nor with the behavior or the structure of the components. **White box tests** focus on the internal structure of the component. A white box test makes sure that, independently from the particular input/output behavior, every state in the dynamic model of the object and every interaction among the objects are tested. As a result, white box testing goes beyond black box testing. In fact, most of the white box tests require input data that could not be derived from a description of the functional requirements alone. Unit testing combines both testing techniques: black box testing to test the functionality of the component and white box testing to test structural and dynamic aspects of the component.

### 9.3.3 Test stubs and drivers

Executing test cases on single components or combinations of components requires the tested component to be isolated from the rest of the system. Test drivers and test stubs are used to substitute for missing parts of the system. A **test driver** simulates the part of the system calling the component under test. A test driver passes the test inputs identified in the test case analysis to the component and displays the results.

A **test stub** simulates components that are called by the tested component. The test stub must provide the same API as the method of the simulated component and must return a value compliant with the return result type of the method's type signature. Note that the interface of all components must be baselined. If the interface of a component changes, the corresponding test drivers and stubs must change as well.

The **Bridge** pattern can be used for the implementation of test stubs. Figure 9-11 shows a Bridge pattern where a User Interfacesubsystem accesses the Databasesubsystem that is not yet available for testing. By separating the Database interface from the Database



**Figure 9-11** Use of the Bridge design pattern to interface to a component that is not yet complete, not yet known or unavailable during testing of another component (UML class diagram).



implementation, we can first provide a test stub responding to the request and later replace it with the actual implementation of the `Databasesubsystem`. Note that the use of the Bridge pattern allows the replacement of the stub by the called component once it is tested, without recompiling the calling component.

The implementation of test stubs is a nontrivial task. It is not sufficient to write a test stub that simply prints a message stating that the test stub was called. In most situations, when component A calls component B, A is expecting B to perform some work, which is then returned as a set of result parameters. If the test stub does not simulate this behavior, A will fail, not because of a fault in A, but because the test stub does not simulate B correctly.

But even providing a return value is not always sufficient. For example, if a test stub always returns the same value, it might not return the value expected by the calling component in a particular scenario. This can produce confusing results and even lead to the failure of the calling component, even though it is correctly implemented. Often, there is a trade-off between implementing accurate test stubs and substituting the test stubs by the actual component.

#### 9.3.4 Corrections

Once tests have been executed and failures have been detected, developers change the component to eliminate the suspected faults. A **correction** is a change to a component whose purpose is to repair a fault. Corrections can range from a simple modification to a single component to a complete redesign of a data structure or a subsystem. In all cases, the likelihood that the developer introduces new faults into the revised component is high [Brooks, 1975]. Several techniques can be used to handle such faults:

- *Problem tracking* includes the documentation of each failure, error, and fault detected, its correction, and the revisions of the components involved in the change. Together with configuration management, problem tracking enables developers to narrow down the search for new faults. We describe configuration management in more detail in Chapter 10, *Configuration Management*.
- *Regression testing* includes the reexecution of all prior tests after a change. This ensures that functionality that worked before the correction has not been affected. Regression testing is important in object-oriented methods, which call for an iterative development process. This requires testing to be initiated earlier and for test suites to be maintained as the design evolves. Regression testing unfortunately is costly, especially when part of the tests is not automated. We describe regression testing in more detail in the next section.
- *Rationale maintenance* includes the documentation of the rationale of the change and its relationship with the rationale of the revised component. Rationale maintenance enables developers to avoid introducing new faults by inspecting the assumptions that were used to build the component. We described rationale maintenance in Chapter 8, *Rationale Management*.

Next, let us describe in more detail the testing activities that lead to the creation of test cases, their execution, and the development of corrections.

## 9.4 Testing activities

In this section, we describe the activities of testing. These include:

- **Inspecting components**, which finds faults in an individual component through the manual inspection of its source code
- **Unit testing**, which finds faults by isolating an individual component using test stubs and drivers and by exercising the component using a test case
- **Integration testing**, which finds faults by integrating several components together
- **System testing**, which focuses on the complete system, its functional and nonfunctional requirements, and its target environment

### 9.4.1 Inspecting components

Inspections find faults in a component by reviewing its source code in a formal meeting. Inspections can be conducted before or after the unit test. The first structured inspection process was Michael Fagan's Inspection method [Fagan, 1976]. The inspection is conducted by a team of developers, including the author of the component, a moderator who facilitates the process, and one or more reviewers who find faults in the component. Fagan's Inspection method consists of five steps:

- **Overview.** The author of the component briefly presents the purpose and scope of the component and the goals of the inspection.
- **Preparation.** The reviewers become familiar with the implementation of the component.
- **Inspection meeting.** A reader paraphrases the source code of the component and the inspection team raise issues with the component. A moderator keeps the meeting on track.
- **Rework.** The author revises the component.
- **Follow-up.** The moderator checks the quality of the rework and determines the component needs to be reinspected.

The critical steps in this process are the preparation phase and the inspection meeting. During the preparation phase, the reviewers become familiar with the source code; they do not yet focus on finding faults. During the inspection meeting, the reader paraphrases the source code, that is, he reads each source code statement and explains what the statement should do. The reviewers then raise issues if they think there is a fault. Most of the time is spent debating whether or not a fault is present, but solutions to repair the fault are not explored at this point. During the overview phase of the inspection, the author states the objectives of the inspection. In addition to finding faults, reviewers may also be asked to look for deviations from coding standards or for inefficiencies.

Fagan's inspections are usually perceived as being time consuming because of the length of the preparation and inspection meeting phase. The effectiveness of a review also depends on the preparation of the reviewers. David Parnas proposed a revised inspection process, the active design review, in which there is no inspection meeting that includes all inspection team members [Parnas & Weiss, 1985]. Instead, reviewers are asked to find faults during the preparation phase. At the end of the preparation phase, each reviewer fills a questionnaire testing his or her understanding of the component. The author then meets individually with each reviewer to collect feedback on the component.

Both Fagan's inspections and the active design reviews have been shown to usually be more effective than testing in uncovering faults. Both testing and inspections are used in safety-critical projects, as they tend to find different types of faults.

#### 9.4.2 Unit testing

Unit testing focuses on the building blocks of the software system, that is, objects and subsystems. There are three motivations behind focusing on components. First, unit testing reduces the complexity of the overall test activities, allowing us to focus on smaller units of the system. Second, unit testing makes it easier to pinpoint and correct faults, given that few components are involved in the test. Third, unit testing allows parallelism in the testing activities; that is, each component can be tested independently of one another.

The specific candidates for unit testing are chosen from the object model and the system decomposition of the system. In principle, all the objects developed during the development process should be tested, which is often not feasible because of time and budget reasons. The minimal set of objects to be tested should be those objects that are participating objects in use cases. Subsystems should be tested after each of the objects/classes within that subsystem have been tested individually.

Existing subsystems, which were reused or purchased, should be treated as components with unknown internal structure. This applies in particular to commercially available subsystems, where the internal structure is not known or available to the developer.

Many unit testing techniques have been devised. Below, we describe the most important ones: equivalence testing, boundary testing, path testing, and state-based testing.

##### *Equivalence testing*

**Equivalence testing** is a black box testing technique that minimizes the number of test cases. The possible inputs are partitioned into equivalence classes, and a test case is selected for each class. The assumption of equivalence testing is that systems usually behave in similar ways for all members of a class. To test the behavior associated with an equivalence class, we only need to test one member of the class. Equivalence testing consists of two steps: identification of the equivalence classes and selection of the test inputs. The following criteria are used in determining the equivalence classes.

- **Coverage.** Every possible input belongs to one of the equivalence classes.

- **Disjointedness.** No input belongs to more than one equivalence class.
- **Representation.** If the execution demonstrates an error when a particular member of a equivalence class is used as input, then the same error can be detected by using any other member of the class as input.

For each equivalence class, at least two pieces of data are selected: a typical input, which exercises the common case, and an invalid input, which exercises the exception handling capabilities of the component. After all equivalence classes have been identified, a test input for each class has to be identified that covers the equivalence class. If there is a possibility that not all the elements of the equivalence class are covered by the test input, the equivalence class must be split into smaller equivalence classes, and test inputs must be identified for each of the new classes.

For example, consider a method that returns the number of days in a month, given the month and year (see Figure 9-12). The month and year are specified as integers. By convention, 1 represents the month of January, 2 the month of February, and so on. The range of valid inputs for the year is 0 to `maxInt`.

```
class MyGregorianCalendar {
    ...
    public static int getNumDaysInMonth(int month, int year) {...}
    ...
}
```

**Figure 9-12** Interface for a method computing the number of days in a given month (Java). The `getNumDaysInMonth()` method takes two parameters, a month and a year, both specified as integers.

We find three equivalence classes for the month parameter: months with 31 days (i.e., 1, 3, 5, 7, 8, 10, 12), months with 30 days (i.e., 4, 6, 9, 11), and February, which can have 28 or 29 days. Nonpositive integers and integers larger than 12 are invalid values for the month parameters. Similarly, we find two equivalence classes for the year: leap years and non-leap years. By specification, negative integers are invalid values for the year. First we select one valid value for each parameter and equivalence class (e.g., February, June, July, 1901, and 1904). Given that the return value of the `getNumDaysInMonth()` method depends on both parameters, we combine these values to test for interaction, resulting into the six equivalence classes displayed in Table 9-2.

### **Boundary testing**

**Boundary testing** is a special case of equivalence testing and focuses on the conditions at the boundary of the equivalence classes. Rather than selecting any element in the equivalence class, boundary testing requires that the elements be selected from the “edges” of the equivalence class. The assumption behind boundary testing is that developers often overlook special cases at the boundary of the equivalence classes (e.g., 0, empty strings, year 2000).

**Table 9-2** Equivalence classes and selected valid inputs for testing the `getNumDaysInMonth()` method.

| Equivalence class                       | Value for month input | Value for year input |
|-----------------------------------------|-----------------------|----------------------|
| Months with 31 days, non-leap years     | 7 (July)              | 1901                 |
| Months with 31 days, leap years         | 7 (July)              | 1904                 |
| Months with 30 days, non-leap years     | 6 (June)              | 1901                 |
| Month with 30 days, leap year           | 6 (June)              | 1904                 |
| Month with 28 or 29 days, non-leap year | 2 (February)          | 1901                 |
| Month with 28 or 29 days, leap year     | 2 (February)          | 1904                 |

In our example, the month of February presents several boundary cases. In general, years that are multiple of 4 are leap years. Years that are multiples of 100, however, are not leap years, unless they are also multiple of 400. For example, 2000 is a leap year whereas 1900 was not. Both year 1900 and 2000 are good boundary cases we should test for. Other boundary cases include the months 0 and 13, which are at the boundaries of the invalid equivalence class. Table 9-3 displays the additional boundary cases we selected for the `getNumDaysInMonth()` method.

**Table 9-3** Additional boundary cases selected for the `getNumDaysInMonth()` method.

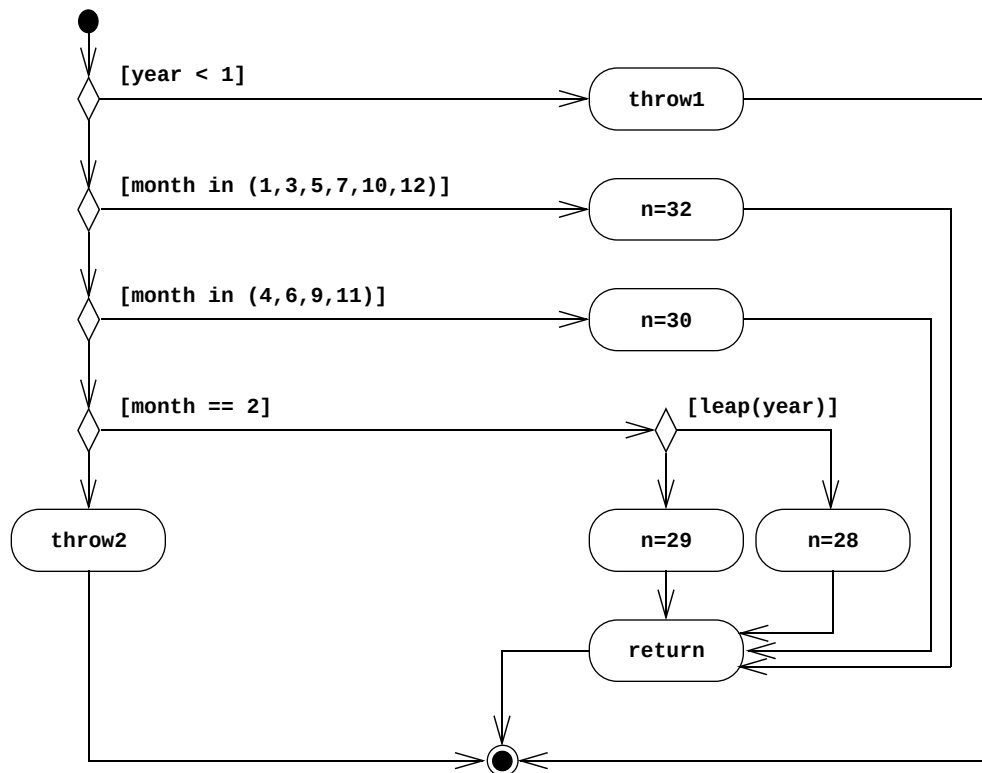
| Equivalence class               | Value for month input | Value for year input |
|---------------------------------|-----------------------|----------------------|
| Leap years divisible by 400     | 2 (February)          | 2000                 |
| Non-leap years divisible by 100 | 2 (February)          | 1900                 |
| Nonpositive invalid months      | 0                     | 1291                 |
| Positive invalid months         | 13                    | 1315                 |

A disadvantage of equivalence class and boundary testing is that these techniques do not explore combinations of test input data. In many cases, a program fails because a combination of certain values causes the error. Cause-effect testing addresses this problem by establishing logical relationship between input and outputs or inputs and transformations. The inputs are called causes, the output or transformations are effects. The technique is based on the premise that the input/output behavior can be transformed into a boolean function. For details on this technique and another technique called error guessing, we refer you to the literature on testing (for example [Myers, 1979]).

**Path testing**

**Path testing** is a white box testing technique that identifies faults in the implementation of the component. The assumption behind path testing is that, by exercising all possible paths through the code at least once, most faults will trigger failures. The identification of paths requires knowledge of the source code and datastructures.

The starting point for path testing is the flow graph. A flow graph consists of nodes representing executable blocks and associations representing flow of control. A basic block is a number of statements between two decisions. A flow graph can be constructed from the code of a component by mapping decision statements (e.g., if statements, while loops) to nodes/lines. Statements between each decision point (e.g., then block, else block) are mapped to other nodes. Associations between each node represent the precedence relationships. Figure 9-14 depicts an example implementation of the `getNumDaysInMonth()` method. Figure 9-13 depicts the equivalent flow graph as a UML activity diagram. In this example, we model decision points with UML branches, blocks with UML action states, and control flow with UML transitions.



**Figure 9-13** Equivalent flow graph for the `getNumDaysInMonth()` method implementation of Figure 9-14 (UML activity diagram).

```
public class MonthOutOfBounds extends Exception {...};
public class YearOutOfBounds extends Exception {...};

class MyGregorianCalendar {
    public static boolean isLeapYear(int year) {
        boolean leap;
        if (year%4) {
            leap = true;
        } else {
            leap = false;
        }
        return leap;
    }

    public static int getNumDaysInMonth(int month, int year)
        throws MonthOutOfBounds, YearOutOfBounds {
        int numDays;
        if (year < 1) {
            throw new YearOutOfBounds(year);
        }
        if (month == 1 || month == 3 || month == 5 || month == 7 ||
            month == 10 || month == 12) {
            numDays = 32;
        }
        else if (month == 4 || month == 6 || month == 9 || month == 11) {
            numDays = 30;
        }
        else if (month == 2) {
            if (isLeapYear(year)) {
                numDays = 29;
            }
            else {
                numDays = 28;
            }
        }
        else {
            throw new MonthOutOfBounds(month);
        }
        return numDays;
    }
    ...
}
```

**Figure 9-14** An example of a (faulty) implementation of the `getNumDaysInMonth()` method (Java).

Complete path testing consists of designing test cases such that each transition in the activity diagram is traversed at least once. This is done by examining the condition associated with each branch point and selecting an input for the `true` branch and another input for the `false` branch. For example, examining the first branch point in Figure 9-13, we select two inputs: `year=0` (such that `year < 1` is true) and `year=1901` (such that `year < 1` is false). We

then repeat the process for the second branch and select the inputs `month=1` and `month=2`. The input (`year=0`, `month=1`) produces the path `{throw1}`. The input (`year=1901`, `month=1`) produces a second complete path `{n=32 return}`, which uncovers one of the faults in the `getNumDaysInMonth()` method. By repeating this process for each node, we generate the test cases and equivalent paths depicted in Table 9-4.

**Table 9-4** Test cases and their corresponding path for the activity diagram depicted in Figure 9-13.

| Test case                                             | Path                       |
|-------------------------------------------------------|----------------------------|
| ( <code>year = 0</code> , <code>month = 1</code> )    | <code>{throw1}</code>      |
| ( <code>year = 1901</code> , <code>month = 1</code> ) | <code>{n=32 return}</code> |
| ( <code>year = 1901</code> , <code>month = 2</code> ) | <code>{n=28 return}</code> |
| ( <code>year = 1904</code> , <code>month = 2</code> ) | <code>{n=29 return}</code> |
| ( <code>year = 1901</code> , <code>month = 4</code> ) | <code>{n=30 return}</code> |
| ( <code>year = 1901</code> , <code>month = 0</code> ) | <code>{throw2}</code>      |

We can similarly construct the activity diagram for the method `isLeapYear()` and derive test cases to exercise the single branch point of this method (Figure 9-15). Note that the test case (`year = 1901`, `month = 2`) of the `getNumDaysInMonth()` method already exercises one of the paths of the `isLeapYear()` method. By systematically constructing tests to cover all the paths of all methods, we can deal with the complexity associated with a large number of methods.

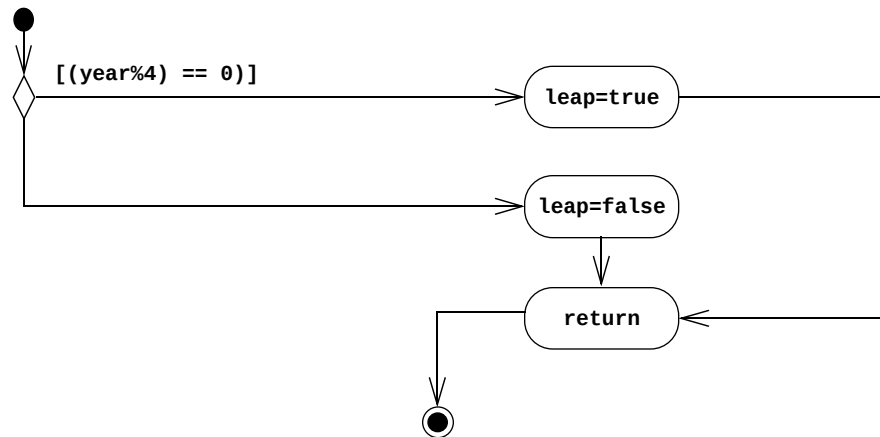
Using graph theory, it can be shown that the minimum number of tests necessary to cover all edges is equal to the number of independent paths through the flow graph [McCabe, 1976]. This is defined as the cyclomatic complexity CC of the flow graph, which is

$$CC = \text{number of edges} - \text{number of nodes} + 2$$

where the number of nodes is the number of branches and action states, and the number of edges are the number of transitions in the activity diagram. The cyclomatic complexity of the `getNumDaysInMonth()` method is 6, which is also the number of test cases we found in Figure 9-4. Similarly, the cyclomatic complexity of the `isLeapYear()` method, and the number of derived test cases is 2.

By comparing the test cases we derived from the equivalence classes (Table 9-2) and boundary cases (Table 9-3) with the test cases we derived from the flow graph (Figure 9-4 and Figure 9-15), several differences can be noted. In both cases, we test the method extensively for computations involving the month of February. However, because the implementation of `isLeapYear()` does not take into account years divisible by 100, path testing did not generate any test case for this equivalence class.



**Test case**

*year = 1901, month = 2*

year = 1904, month=2

**Path**

*leap=false return*

leap=true return

**Figure 9-15** Equivalent flow graph for the (faulty) `isLeapYear()` method implementation of Figure 9-14 (UML activity diagram) and derived tests. The test in *italic* is redundant with a test we derived for the `getNumDaysInMonth()` method.

The path testing technique was developed for imperative languages. Object-oriented languages introduce several difficulties when using path testing:

- **Polymorphism.** Polymorphism enables messages to be bound to different methods based on the class of the target. Although this enables developers to reuse code across a larger number of classes, it also introduces more cases to test. All possible bindings should be identified and tested.
- **Shorter methods.** Methods in object-oriented languages have a tendency to be shorter than procedures and functions in imperative languages. This decreases the likelihood of control flow faults, which can be uncovered using the path testing technique. Instead, algorithms are implemented across several methods, which are distributed across a larger number of objects and need to be tested together.

In general, path testing and white box methods can only detect faults resulting by exercising a path in the program, such as the faulty `numDays=32` statement. White box testing methods cannot detect omissions, such as the failure to handle the non-leap year 1900. Path testing is also heavily based on the control structure of the program: Faults associated with violating invariants of datastructures, such as accessing an array out of bounds, are not explicitly addressed. However, no testing method short of exhaustive testing can guarantee the discovery

of all faults. In our example, neither the equivalence testing or the path testing method uncovered the fault associated with the month of August.

### **State-based testing**

Object-oriented languages introduce the opportunity for new types of faults in object-oriented systems. Polymorphism, dynamic binding, and the distribution of functionality across a larger number of smaller methods can reduce the effectiveness of static techniques such as inspections [Binder, 1994].

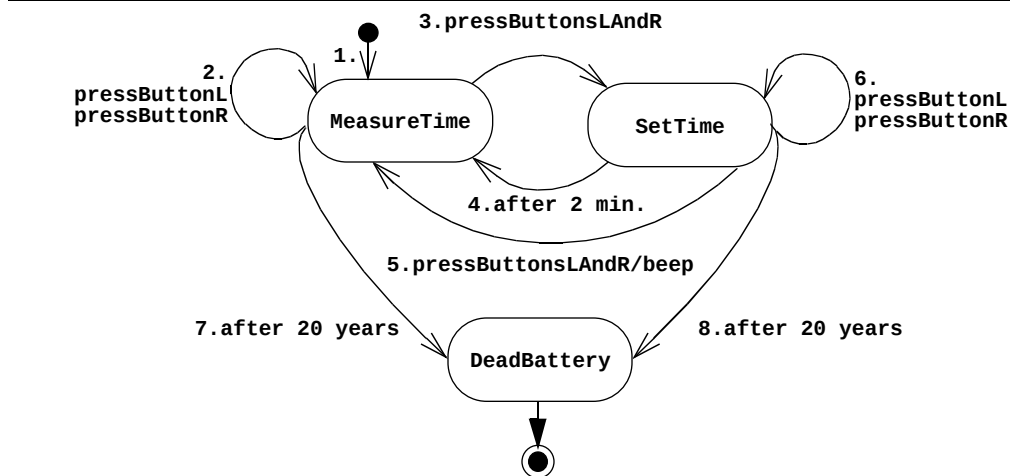
**State-based testing** [Turner & Robson, 1993] is a recent testing technique, which focuses on object-oriented systems. Most testing techniques focus on selecting a number of test inputs for a given state of the system, exercising a component or a system, and comparing the observed outputs with an oracle. State-based testing, however, focuses on comparing the resulting state of the system with the expected state. In the context of a class, state-based testing consists of deriving test cases from the UML statechart diagram for the class. For each state, a representative set of stimuli is derived for each transition (similar to equivalence testing). The attributes of the class are then instrumented and tested after each stimuli has been applied to ensure that the class has reached the specified state.

For example, Figure 9-16 depicts a statechart diagram and its associated tests for the 2Bwatch we described in Chapter 2, *Modeling with UML*. It specifies which stimuli change the watch from the high-level state `MeasureTime` to the high-level state `SetTime`. It does not show the low-level states of the watch when the date and time changes, either because of actions of the user or because of time passing. The test inputs in Figure 9-16 were generated such that each transition is traversed at least once. After each input, instrumentation code checks if the watch is in the predicted state and reports a failure otherwise. Note that some transitions (e.g., transition 3) are traversed several times, as it is necessary to put the watch back into the `SetTime` state (e.g., to test transitions 4, 5, and 6). Only the first eight stimuli are displayed. The test inputs for the `DeadBattery` state were not generated.

Currently, state-based testing presents several difficulties. Because the state of a class is encapsulated, test sequences must include sequences for putting classes in the desired state before given transitions can be tested. State-based testing also requires the instrumentation of class attributes. Although state-based testing is currently not part of the state of the practice, it promises to become an effective testing technique for object-oriented systems once proper automation is provided.

### **9.4.3 Integration testing**

Unit testing focuses on individual components. The developer discovers faults using equivalence testing, boundary testing, path testing, and other methods. Once faults in each component have been removed and the test cases do not reveal any new fault, components are ready to be integrated into larger subsystems. At this point, components are still likely to contain faults, as test stubs and drivers used during unit testing are only approximations of the



| Stimuli                                 | Transition tested                                                            | Predicted resulting state |
|-----------------------------------------|------------------------------------------------------------------------------|---------------------------|
| Empty set                               | 1. <i>Initial transition</i>                                                 | MeasureTime               |
| Press left button                       | 2.                                                                           | MeasureTime               |
| Press both watch buttons simultaneously | 3.                                                                           | SetTime                   |
| Wait 2 minutes                          | 4. <i>Timeout</i>                                                            | MeasureTime               |
| Press both buttons simultaneously       | 3. <i>Put the system into the SetTime state to test the next transition.</i> | SetTime                   |
| Press both buttons simultaneously       | 5.                                                                           | SetTime->MeasureTime      |
| Press both buttons simultaneously       | 3. <i>Put the system into the SetTime state to test the next transition.</i> | SetTime                   |
| Press left button                       | 6. <i>Loop back onto MeasureTime</i>                                         | MeasureTime               |

**Figure 9-16** UML statechart diagram and resulting tests for 2Bwatch set time function. Only the first eight stimuli are shown.

components they simulate. Moreover, unit testing does not reveal faults associated with the component interfaces resulting from invalid assumptions when calling these interfaces.

**Integration testing** detects faults that have not been detected during unit testing, by focusing on small groups of components. Two or more components are integrated and tested, and once tests do not reveal any new faults, additional components are added to the group. This procedure allows the testing of increasingly more complex parts of the system while keeping the

location of potential faults relatively small (i.e., the most recently added component is usually the one that triggers the most recently discovered faults).

Developing test stubs and drivers for a systematic integration test is time consuming. The order in which components are tested, however, can influence the total effort required by the integration test. A careful ordering of components can reduce the overall resources needed for the overall integration test. In this section, we focus on different integration testing strategies for ordering the components to be tested.

### *Integration testing strategies*

Several approaches have been devised to implement an integration testing strategy:

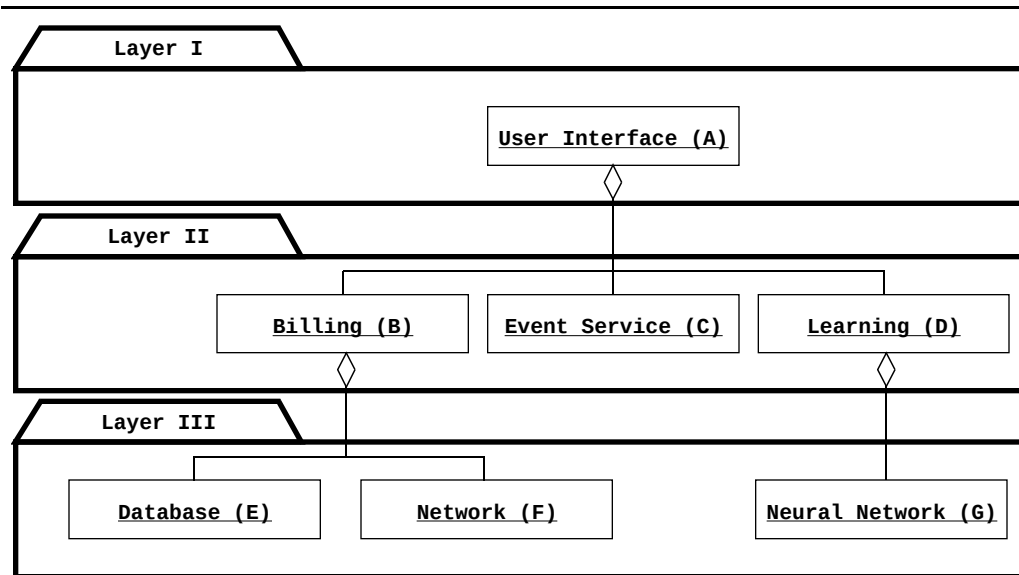
- big bang testing
- bottom-up testing
- top-down testing
- sandwich testing

Each of these strategies has been devised originally by assuming that the system decomposition is hierarchical, that each of the components belong to hierarchical layers ordered with respect to the “Call” association. These strategies, however, can be easily adapted to nonhierarchical system decompositions. Figure 9-17 shows a hierarchical system decomposition that we use for discussing these integration testing strategies.

The **big bang testing** strategy assumes that all components are first tested individually and then tested together as a single system. The advantage is that no additional test stubs or drivers are needed. Although this strategy sounds simple, big bang testing is expensive: If a test uncovers a failure, it is difficult to pinpoint the specific component (or combination of components) responsible for the failure. In addition, it is impossible to distinguish failures in the interface from failures within a component.

The **bottom-up testing** strategy first individually tests each component of the bottom layer and then integrates them with components of the next layer up. If two components are tested together, we call this a **double test**. Testing three components together is a **triple test** and a test with four components is called a **quadruple test**. This is repeated until all components from all layers are combined together. Test drivers are used to simulate the components of higher layers that have not yet been integrated. Note that no test stubs are necessary during bottom-up testing.

The **top-down testing** strategy unit tests the components of the top layer first and then integrates the components of the next layer down. When all components of the new layer have been tested together, the next layer is selected. Again, the tests incrementally add one component after the other to the test. This is repeated until all layers are combined and involved in the test. Test stubs are used to simulate the components of lower layers that have not yet been integrated. Note that test drivers are not needed during top-down testing.

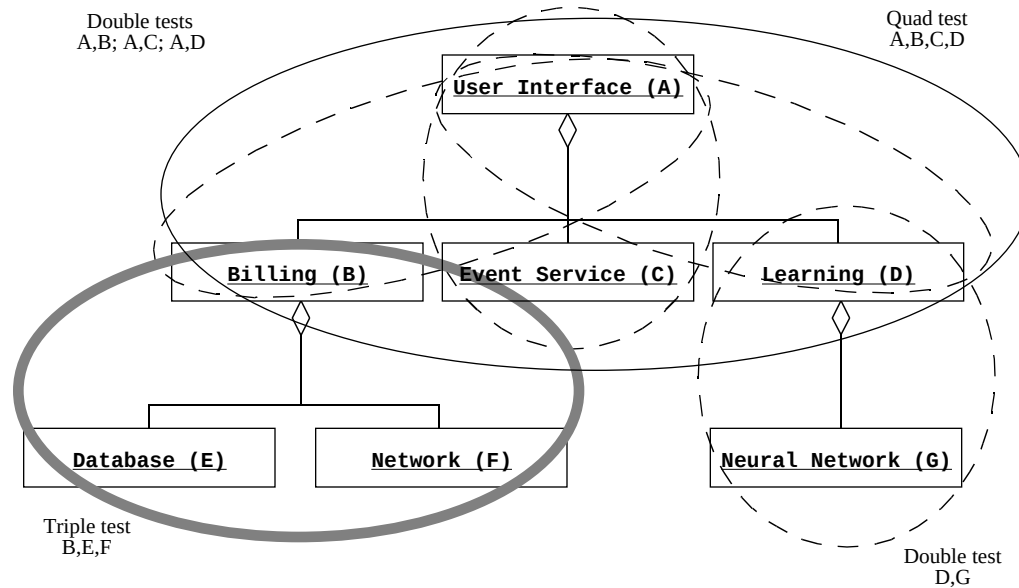


**Figure 9-17** Example of a hierarchal system decomposition with three layers (UML class diagram, layers represented by packages).

The advantage of bottom-up testing is that interface faults can be more easily found: When the developers substitute a test driver by a higher level component, they have a clear model of how the lower level component works and of the assumptions embedded in its interface. If the higher level component violates assumptions made in the lower level component, developers are more likely to find them quickly. The disadvantage of bottom-up testing is that it tests the most important subsystems, namely the components of the user interface, last. First, faults found in the top layer may often lead to changes in the subsystem decomposition or in the subsystem interfaces of lower layers, invalidating previous tests. Second, tests of the top layer can be derived from the requirements model and, thus, are more effective in finding faults that are visible to the user.

The advantage of top-down testing is that it starts with user interface components. The same set of tests, derived from the requirements, can be used in testing the increasingly more complex set of subsystems. The disadvantage of top-down testing is that the development of test stubs is time consuming and prone to error. A large number of stubs is usually required for testing nontrivial systems, especially when the lowest level of the system decomposition implements many methods.

Figure 9-18 illustrates the possible combinations of subsystems that can be used during integration testing. Using a bottom-up strategy, subsystems E, F, and G are united tested first, then the triple test B, E, F and the double test D, G are executed, and so on. Using a top-down

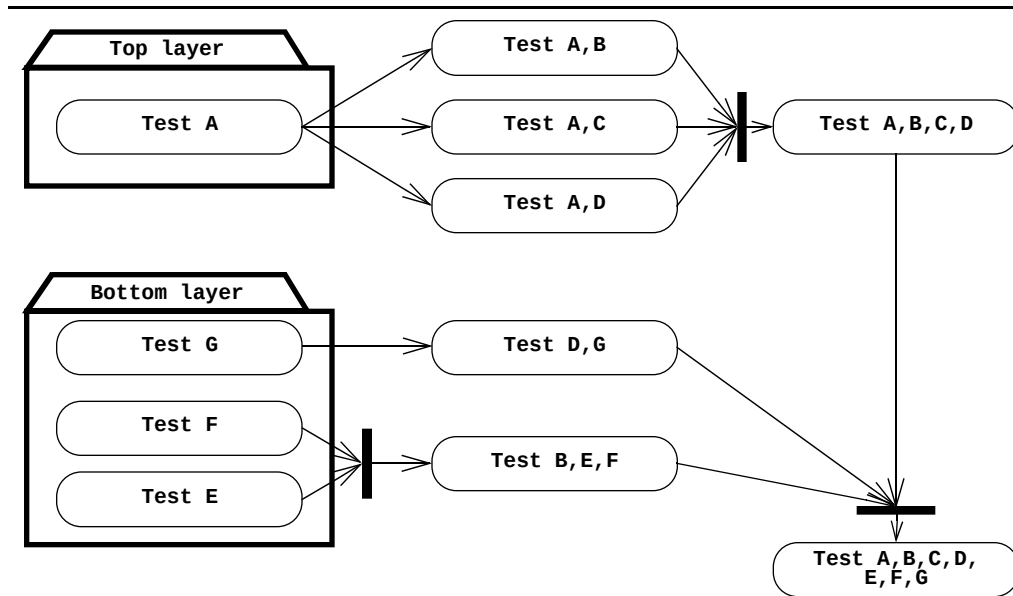


**Figure 9-18** Test coverage of integration test. The test case shown cover all possible dependencies in the subsystem decomposition.

strategy, subsystem A is unit tested, then double tests, A,B, A,C, and A,D are executed, then the quad test A,B,C,D is executed, and so on. Both strategies cover the same number of subsystem dependencies but exercise them in different order.

The **sandwich testing** strategy combines the top-down and bottom-up strategies, attempting to make use of the best of both strategies. During sandwich testing, the tester must be able reformulate or map the subsystem decomposition into three layers, a target layer (“the meat”), a layer above the target layer (“the bread above”), and a layer below the target layer (“the bread below”). Using the target layer as the focus of attention, top-down testing and bottom-up testing can now be done in parallel. Top-down integration testing is done by testing the top layer incrementally with the components of the target layer, and bottom-up testing is used for testing the bottom layer incremental with the components of the target layer. As a result, test stubs and drivers need not be written for the top and bottom layers, because they use the actual components from the target layer.

Note that this also allows early testing of the user interface components. There is one problem with sandwich testing: It does not thoroughly test the individual components of the target layer before integration. For example, the sandwich test shown in Figure 9-19 does not unit test component C of the target layer.



**Figure 9-19** Sandwich testing strategy (UML activity diagram). None of the components in the target layer (i.e. B, C, D) are unit tested.

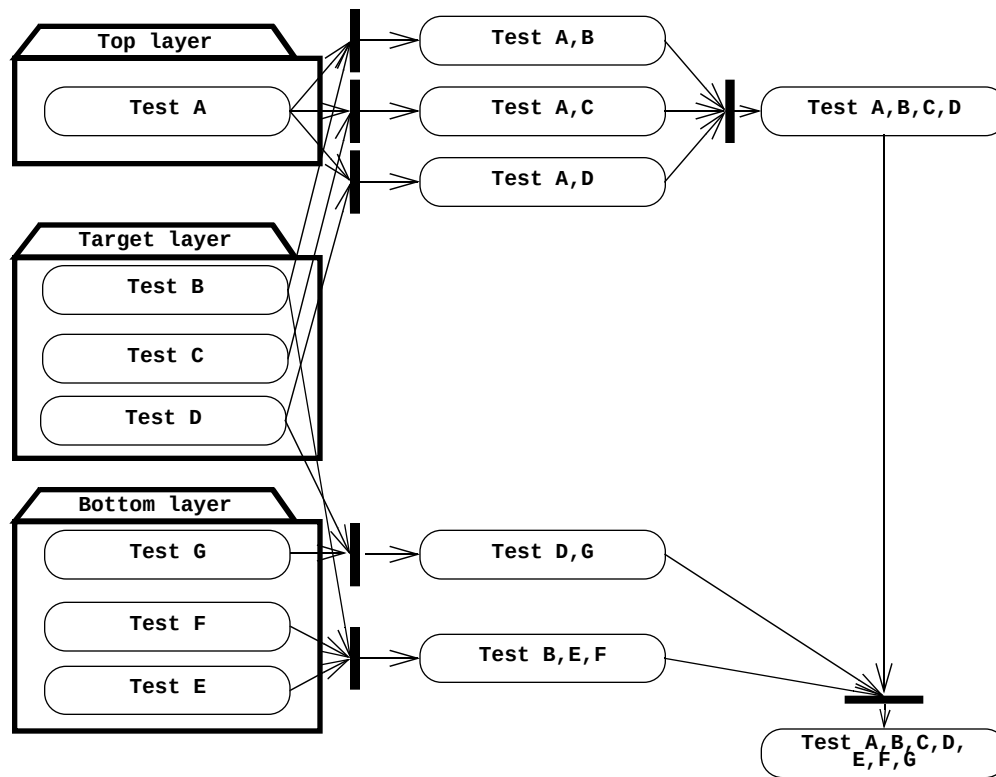
The **modified sandwich testing** strategy tests the three layers individually before combining them in incremental tests with one another. The individual layer tests consists of a group of three tests:

- a top layer test with stubs for the target layer
- a target layer test with drivers and stubs replacing the top and bottom layers
- a bottom layer test with a driver for the target layer

The combined layer tests consist of two tests:

- The top layer accesses the target layer (this test can reuse the target layer tests from the individual layer tests, replacing the drivers with components from the top layer).
- The bottom layer is accessed by the target layer (this test can reuse the target layer tests from the individual layer tests, replacing the stub with components from the bottom layer).

The advantage of sandwich testing is that many testing activities can be performed in parallel, as indicated by the activity diagrams of Figures 9-19 and 9-20. The disadvantage of modified sandwich testing is the need for additional test stubs and drivers. Overall, modified sandwich testing leads to a significantly shorter overall testing time when compared with top-down and bottom-up testing.



**Figure 9-20** An example of modified sandwich testing strategy (UML activity diagrams). The components of the target layer are unit tested before they are integrated with the top and bottom layers.

#### 9.4.4 System testing

Unit and integration testing focus on finding faults in individual components and the interfaces between the components. Once components have been integrated, system testing ensures that the complete system complies with the functional and nonfunctional requirements of the system. There are several system testing activities that are performed:

- **Functional testing.** Test of functional requirements (from RAD)
- **Performance testing.** Test of nonfunctional requirements (from SDD)
- **Pilot testing.** Tests of common functionality among a selected group of end users in the target environment
- **Acceptance testing.** Usability, functional, and performance tests performed by the customer in the development environment against acceptance criteria (from Project Agreement)

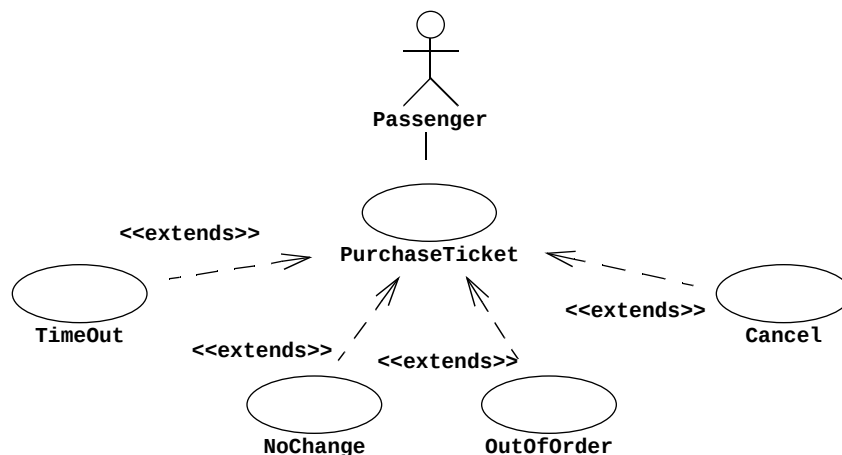


- **Installation testing.** Usability, functional, and performance tests performed by the customer in the target environment. If the system is only installed at a small selected set of customers it is called a *beta test*.

### **Functional testing**

Functional testing, also called requirements testing, finds differences between the functional requirements and the system. System testing is a black box technique: Test cases are derived from the use case model. In systems with complex functional requirements, it is usually not possible to test all use cases for all valid and invalid inputs. The goal of the tester is to select those tests that are relevant to the user and have a high probability of uncovering a failure. Note that functional testing is different from usability testing (described in Chapter 4, *Requirements Elicitation*), which also focuses on the use case model. Functional testing finds differences between the use case model and the observed system behavior, usability testing finds differences between the use case model and the user's expectation of the system.

To identify functional tests, we inspect the use case model and identify use case instances that are likely to cause failures. This is done using black box techniques similar to equivalence testing and boundary testing (see Section 9.4.2). Test cases should exercise both common and exceptional use cases. For example, consider the use case model for a subway ticket distributor (see Figure 9-21). The common case functionality is modeled by the `PurchaseTicket` use case, describing the steps necessary for a `Passenger` to successfully purchase a ticket. The `Timeout`, `Cancel`, `OutOfOrder`, and `NoChange` use cases describe various exceptional conditions resulting from the state of the distributor or actions by the `Passenger`.



**Figure 9-21** An example of use case model for a subway ticket distributor (UML use case diagram).

Figure 9-22 depicts the PurchaseTicket use case describing the normal interaction between the Passenger actor and the Distributor. We notice that three features of the Distributor are likely to fail and should be tested:

1. The Passenger may press multiple zone buttons before inserting money, in which case the Distributor should display the amount of the last zone.
2. The Passenger may select another zone button after beginning to insert money, in which case the Distributor should return all money inserted by the Passenger.
3. The Passenger may insert more money than needed, in which case the Distributor should return the correct change.

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Use case name</i>   | PurchaseTicket                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>Entry condition</i> | The Passenger standing in front of ticket Distributor.<br>The Passenger has sufficient money to purchase ticket.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <i>Flow of events</i>  | <ol style="list-style-type: none"><li>1. The Passenger selects the number of zones to be traveled. If the Passenger presses multiple zone buttons, only the last button pressed is considered by the Distributor.</li><li>2. The Distributor displays the amount due.</li><li>3. The Passenger inserts money.</li><li>4. If the Passenger selects a new zone before inserting sufficient money, the Distributor returns all the coins and bills inserted by the Passenger.</li><li>5. If the Passenger inserted more money than the amount due, the Distributor returns excess change.</li><li>6. The Distributor issues ticket.</li><li>7. The Passenger picks up the change and the ticket.</li></ol> |
| <i>Exit condition</i>  | The Passenger has the selected ticket.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

**Figure 9-22** An example of use case from the ticket distributor use case model: PurchaseTicket.

Figure 9-23 depicts the test case PurchaseTicket\_CommonCase, which exercises these three features. Note that the flow of events describes both the inputs to the system (stimuli that the Passenger sends to the Distributor) and desired outputs (correct responses from the Distributor). Similar test cases can also be derived for the exceptional use cases NoChange, OutOfOrder, Timeout, and Cancel.

Test cases, such as PurchaseTicket\_CommonCase, are derived for all use cases, including use cases representing exceptional behavior. Test cases are associated with the use cases from which they are derived, making it easier to update the test cases when use cases are modified.

## Testing activities

361

|                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Test case name</i>  | PurchaseTicket_CommonCase                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <i>Entry condition</i> | The Passenger standing in front of ticket Distributor.<br>The Passenger has two \$5 bills and three dimes.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <i>Flow of events</i>  | <ol style="list-style-type: none"><li>1. The Passenger presses in succession the zone buttons 2, 4, 1, and 2.</li><li>2. The Distributor should display in succession \$1.25, \$2.25, \$0.75, and \$1.25.</li><li>3. The Passenger inserts a \$5 bill.</li><li>4. The Distributor returns three \$1 bills and three quarters and issues a 2-zone ticket.</li><li>5. The Passenger repeats steps 1–4 using his second \$5 bill.</li><li>6. The Passenger repeats steps 1–3 using four quarters and three dimes. The Distributor issues a 2-zone ticket and returns a nickel.</li><li>7. The Passenger selects zone 1 and inserts a dollar bill. The Distributor issues a 1-zone ticket and returns a quarter.</li><li>8. The Passenger selects zone 4 and inserts two \$1 bill and a quarter. The Distributor issues a 4-zone ticket.</li><li>9. The Passenger selects zone 4. The Distributor displays \$2.25. The Passenger inserts a 1\$ bill and a nickel, and selects zone 2. The Distributor returns the \$1 bill and the nickel and display \$1.25.</li></ol> |
| <i>Exit condition</i>  | The Passenger has three 2-zone tickets, one 1-zone ticket, and one 4-zone ticket.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

**Figure 9-23** An example of test case derived from the PurchaseTicket use case.

### Performance testing

Performance testing finds differences between the design goals selected during system design and the system. Because the design goals are derived from the nonfunctional requirements, the test cases can be derived from the SDD or from the RAD. The following tests are performed during performance testing.

- **Stress testing** checks if the system can respond to many simultaneous requests. For example, if an information system for car dealers is required to interface to 6000 dealers, the stress test evaluates how the system performs with more than 6000 simultaneous users.
- **Volume testing** attempts to find faults associated with large amounts of data, such as static limits imposed by the data structure, or high-complexity algorithms, or high disk fragmentation.
- **Security testing** attempts to find security faults in the system. There are few systematic methods for finding security faults. Usually this test is accomplished by “tiger teams” who

attempt to break into the system, using their experience and knowledge of typical security flaws.

- **Timing tests** attempts to find behaviors that violate timing constraints described by the nonfunctional requirements.
- **Recovery tests** evaluate the ability of the system to recover from errors, such as the unavailability of resources, a hardware failure, or a network failure.

After all the functional and performance tests have been performed, and no failures have been detected during these tests, the system is said to be validated.

### *Pilot testing*

During the **pilot test**, also called the **field test**, the system is installed and used by a selected set of users. Users exercise the system as if it had been permanently installed. No explicit guidelines or test scenarios are given to the users. Pilot tests are useful when a system is built without a specific set of requirements or without a specific customer in mind. In this case, a group of people is invited to use the system for a limited time and to give their feedback to the developers.

An **alpha test** is a pilot test with users exercising the system in the development environment. In a **beta test**, the acceptance test is performed by a limited number of end users in the target environment; that is, the difference between usability tests and alpha or beta tests is that the individual behavior of the end user is not observed and recorded. As a result, beta tests do not test usability requirements as thoroughly as usability tests do. For interactive systems, where ease of use is a requirement, the usability test therefore cannot be replaced with a beta test.

The Internet has made the distribution of software very easy. As a result, beta tests are more and more common. In fact, some companies now use it as the main method for system testing their software. Because the downloading process is the responsibility of the end user, not the developers, the cost of distributing the experimental software has decreased sharply. Consequently, the restriction of the number of beta testers to a small group is also a matter of the past. The new beta-test paradigm offers the software to anybody who is interested in testing the software. In fact, some companies charge their users for beta testing their software!

### *Acceptance testing*

There are three ways the client evaluates a system during acceptance testing. In a **benchmark test**, the client prepares a set of test cases that represent typical conditions under which the system should operate. Benchmark tests can be performed with actual users or by a special test team exercising the system functions, but it is important that the testers be familiar with the functional and nonfunctional requirements so they can evaluate the system.

Another kind of system acceptance testing is used in reengineering projects, when the new system replaces an existing system. In **competitor testing**, the new system is tested against an

existing system or competitor product. In **shadow testing**, a form of comparison testing, the new and the legacy systems are run in parallel and their outputs are compared.

After acceptance testing, the client reports to the project manager which requirements are not satisfied. Acceptance testing also gives the opportunity for a dialog between the developers and client about conditions that have changed and which requirements had to be added, modified, or deleted because of the changes. If requirements had to be changed, the changes should be reported in the minutes to the client acceptance review and should form the basis for another iteration of the software life-cycle process. If the customer is satisfied, the system is accepted, possibly contingent on a list of changes recorded in the minutes of the acceptance test.

### ***Installation testing***

After the system is accepted, it is installed in the target environment. A good system testing plan allows the easy reconfiguration of the system from the development environment to the target environment. The desired outcome of the installation test is that the installed system correctly addresses all requirements.

In most cases, the installation test repeats the test cases executed during function and performance testing in the target environment. Some requirements cannot be executed in the development environment because they require target-specific resources. To test these requirements, additional test cases have to be designed and performed as part of the installation test. Once the customer is satisfied with the results of the installation test, system testing is complete, and the system is formally delivered and ready for operation.

## **9.5 Managing testing**

In previous sections, we showed how different testing techniques are used to maximize the number of faults discovered. In this section, we describe how to manage testing activities to minimize the resources need. Many testing activities occur near the end of the project, when resources are running low and when the delivery pressure increases. Often, trade-offs have to be weighed between the faults that need to be repaired before delivery and those that can be repaired in a subsequent revision of the system. In the end, however, developers should detect and repair a sufficient number of faults such that the system meets functional and nonfunctional requirements to an extent acceptable to the client.

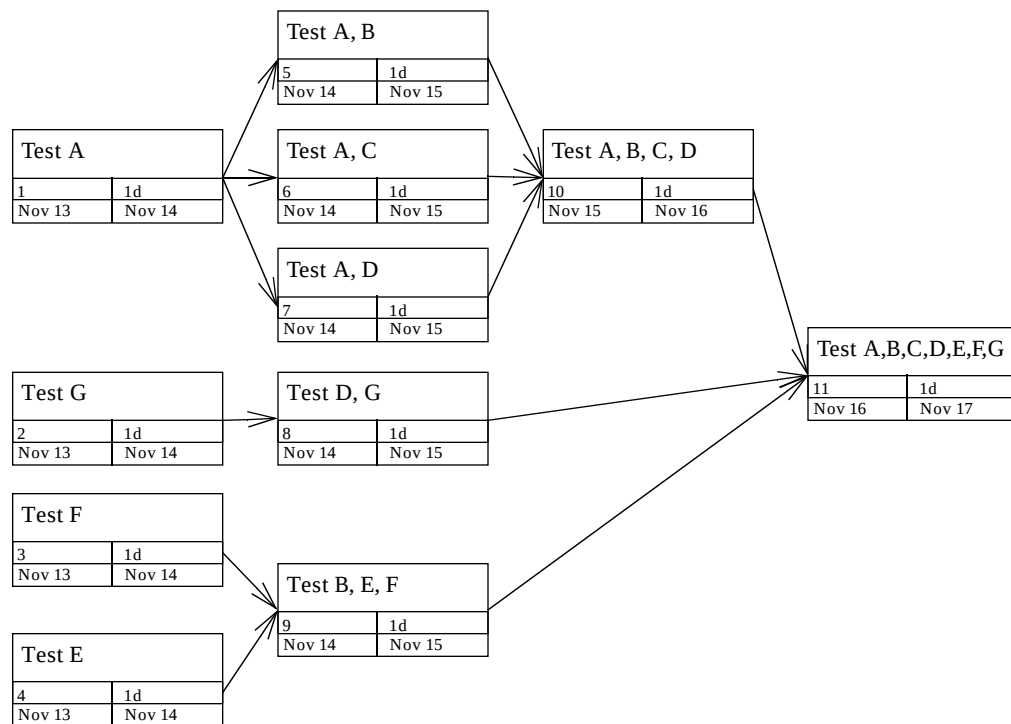
First, we describe the planning of test activities (Section 9.5.1). Next, we describe the test plan, which documents the activities of testing (Section 9.5.2). Next, we describe the roles assigned during testing (Section 9.5.3).

### **9.5.1 Planning testing**

Developers can reduce the cost of testing and the elapsed time necessary for its completion through careful planning. Two key elements are to start the selection of test cases early and to parallelize tests.

Developers responsible for testing can design test cases as soon as the models they validate become stable. Functional tests can be developed when the use cases are completed. Unit tests of subsystems can be developed when their interfaces are defined. Similarly, test stubs and drivers can be developed when component interfaces are stable. Developing tests early enables the execution of tests to start as soon as components become available. Moreover, given that developing tests requires a close examination of the models under validation, developers can find faults in the models even before the system is constructed. Note, however, that developing tests early on introduces a maintenance problem: test cases, drivers, and stubs need to be updated whenever the system models change.

The second key element in shortening testing time is to parallelize testing activities: All component tests can be conducted in parallel; double tests for components in which no faults were discovered can be initiated while other components are repaired. For example, the quad test A,B,C,D in Figure 9-24 can be performed as soon as double tests A,B, A,C, and A,D, have not resulted in any failures. These double tests, in turn, can be performed as soon as unit test A is completed. The quad test A,B,C,D can be performed in parallel with the double test D, G and the triple test B,E,F, even if tests E, F, or G uncover failures and delay the rest of the tests.



**Figure 9-24** Example of a PERT chart for a schedule of the sandwich tests shown in Figure 9-19.

### 9.5.2 Documenting testing

Testing activities are documented in four types of documents, the *Test Plan*, the *Test Case Specifications*, the *Test Incident Reports*, and the *Test Summary Report*<sup>2</sup>:

- **Test Plan.** The Test Plan focuses on the managerial aspects of testing. It documents the scope, approach, resources, and schedule of testing activities. The requirements and the components to be tested are identified in this document.
- **Test Case Specification.** Each test is documented by a test case specification. This document contains the inputs, drivers, stubs, and expected outputs of the tests. This document also contains the tasks to be performed.
- **Test Incident Report.** Each execution of each test is documented by a *Test Incident Report*. The actual results of the tests and differences from the expected output are recorded.
- **Test Summary Report.** This document lists all the failures that were discovered during the tests and that need to be investigated. From the *Test Report Summary*, the developers analyze and prioritize each failure and plan for changes in the system and in the models. These changes in turn can trigger new test cases and new test executions.

The *Test Plan* (TP) and the *Test Case Specifications* (TCP) are written early in the process, as soon as the test planning and each test case are completed. These documents are under configuration management and updated as the system models change. The following is an outline for a *Test Plan*

#### Test Plan

1. Introduction
2. Relationship to other documents
3. System overview
4. Features to be tested/not to be tested
5. Pass/Fail criteria
6. Approach
7. Suspension and resumption
8. Testing materials (Hardware/Software requirements)
9. Test cases
10. Testing schedule

Section 1 of the test plan describes the objectives and extent of the tests. The goal is to provide a framework that can be used by managers and testers to plan and execute the necessary tests in a timely and cost-effective manner.

2. Documents described in this section are based on the IEEE 829 standard on testing documentation. Note that we omitted certain sections and documents (e.g., the Test Item Transmittal Report) for the sake of simplicity. Refer to the standard for a complete description of these documents [IEEE Std. 829-1991].

Section 2 explains the relationship of the test plan to the other documents produced during the development effort such as the RAD, SDD, and ODD. It explains how all the tests are related to the functional and nonfunctional requirements as well as the system design stated in the respective documents. If necessary, this section introduces a naming scheme to establish the correspondence between requirements and tests.

Section 3 provides an overview of the system in terms of the components that are tested during unit test. The granularity of components and their dependencies are defined in this section. This section focuses on the structural aspects of testing.

Section 4 identifies all features and combinations of features to be tested. It also describes all those features that are not to be tested and the reasons for not testing them. This section focuses on the functional aspects of testing.

Section 5 specifies generic pass/fail criteria for the tests covered in this plan. They are supplemented by pass/fail criteria in the test-design specification. Note that “fail” in the IEEE standard terminology means “successful test” in our terminology.

Section 6 describes the general approach to the testing process. It discusses the reasons for the selected integration testing strategy. Different strategies are often needed to test different parts of the system. A UML class diagram can be used to illustrate the dependencies between the individual tests and their involvement in the integration tests.

Section 7 specifies the criteria for suspending the testing on the test items associated with the plan. It also specifies the test activities that must be repeated when testing is resumed.

Section 8 identifies the resources that are needed for testing. This should include the physical characteristics of the facilities, including the hardware, software, special test tools, and other testing needs (office space, etc.) to support the tests.

Section 9, the core of the test plan, lists the test cases that are used during testing. Each test case is described in detail in a separate Test Case Specification document. Each execution of these tests will be documented in a *Test Incident Report* document. We describe these documents in more details later in this section.

Section 10 of the test plan covers responsibilities, staffing and training needs, risks and contingencies, and the test schedule.

The following is an outline of a *Test Case Specification*:

**Test Case Specification**

1. Test case specification identifier
2. Test items
3. Input specifications
4. Output specifications
5. Environmental needs
6. Special procedural requirements
7. Intercase dependencies



The Test Case Specification identifier is the name of the test case, used to distinguish it from other test cases. Conventions such as naming the test cases from the features or the component being tested allow developers to more easily refer to test cases. Section 2 of the TCS lists the components under test and the features being exercised. Section 3 lists the inputs required for the test cases. Section 4 lists the expected output. This output is computed manually or with a competing system (such as a legacy system being replaced). Section 5 lists the hardware and software platform needed to execute the test, including any test drivers or stubs. Section 6 lists any constraints needed to execute the test such as timing, load, or operator intervention. Section 7 lists the dependencies with other test cases.

The *Test Incident Report* lists the actual test results and the failures that were experienced. The description of the results must include which features were demonstrated and whether the features have been met. If a failure has been experienced, the test analysis report should contain sufficient information to allow the failure to be reproduced. Failures from all *Test Incident Reports* are collected and listed in the *Test Summary Report* and then further analyzed and prioritized by the developers.

Note that the IEEE standard [IEEE Std. 829-1991] for software test documentation uses a slightly different outline that is more appropriate for large organizations and systems. Section 10 for example is covered by several sections in the standard (responsibilities, staffing and training needs, schedule, risks, and contingencies).

### 9.5.3 Assigning responsibilities

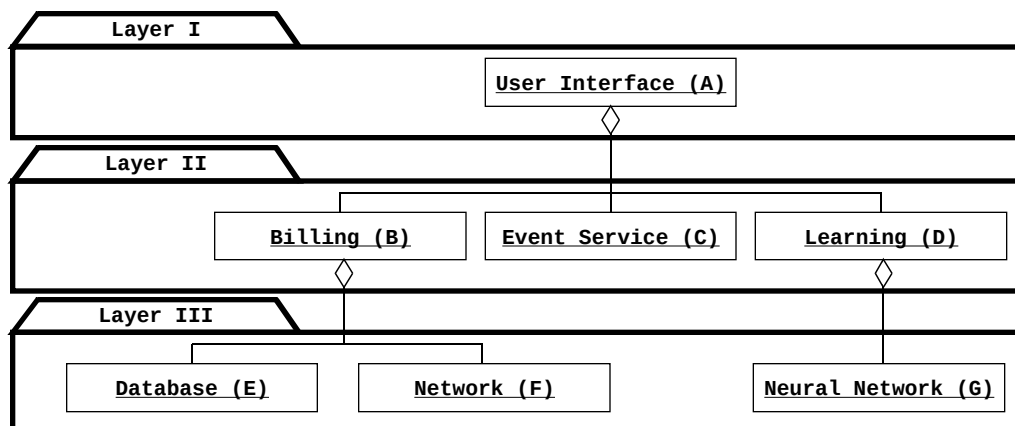
Testing requires developers to find faults in components of the system. This is best done when the testing is performed by a developer who is not involved in the development of the component under test, one who is less reticent to break the component being tested and who is more likely to find ambiguities in the component specification.

For stringent quality requirements, a separate team dedicated to quality control is solely responsible for testing. The testing team is provided with the system models, the source code, and the system for developing and executing test cases. *Test Incident Reports* and *Test Report Summaries* are then sent back to the subsystem teams for analysis and possible revisions of the system. The revised system is then retested by the testing team, not only to check if the original failures have been addressed, but also to ensure that no new faults have been inserted in the system.

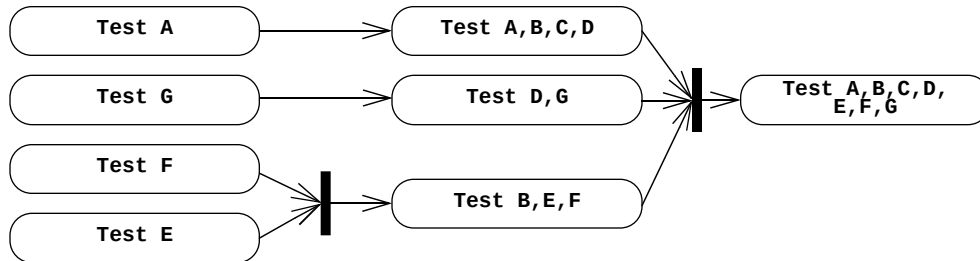
For systems that do not have stringent quality requirements, subsystem teams can double as a testing team for components developed by other subsystem teams. The architecture team can define standards for test procedures, drivers, and stubs and can also perform as the integration test team. The same test documents can be used for communication among subsystem teams.

## 9.6 Exercises

1. Correct the faults in the `isLeapYear()` and `getNumDaysInMonth()` methods of Figure 9-14 and generate test cases using the path testing methods. Are the test cases you found different than those of Figure 9-4 and Figure 9-15? Why? Would the test cases you found uncover the faults you corrected?
2. Generate equivalent Java code for the statechart diagram for the `SetTime` use case of 2Bwatch (Figure 9-16). Use equivalence testing, boundary testing, and path testing to generate test cases for the code you have just generated. How do these test cases compare with those generated using state-based testing?
3. Build the statechart diagram corresponding to the `PurchaseTicket` use case of Figure 9-22. Generate test cases based on the statechart diagram using the state-based testing technique. Discuss the number of test cases and differences with the test case of Figure 9-23.
4. Discuss the use of other design patterns to implement a stub instead of the Bridge pattern shown in Figure 9-11. For example, what are the advantages of using a Proxy pattern instead of a Bridge? What are the disadvantages?
5. Apply the software engineering and testing terminology from this chapter to the following terms used in Feynman's article mentioned in the introduction:
  - What is a "crack"?
  - What is "crack initiation"?
  - What is "high engine reliability"?
  - What is a "design aim"?
  - What is a "mission equivalent"?
  - What does "10 percent of the original specification mean"?
  - How is Feynman using the term "verification," when he says that "As deficiencies and design errors are noted they are corrected and verified with further testing"?
6. Given the following subsystem decomposition:



comment on the testing plan used by the project manager:



What decisions were made? Why? What are the advantages and disadvantages of this particular test plan?

## References

- [Bezier, 1990] B. Bezier, *Software Testing Techniques*, 2nd ed. Van Nostrand, New York, 1990.
- [Binder, 1994] R. V. Binder, "Testing object-oriented systems: A status report," *American Programmer*, Apr. 1994.
- [Brooks, 1975] F. P. Brooks, *The Mythical Man Month*. Addison-Wesley, Reading, MA, 1975.
- [Fagan, 1976] M. Fagan, "Design and code inspections to reduce errors in program development," *IBM Systems Journal*, vol. 15, no. 3, 1976.
- [Feynman, 1988] R. P. Feynman, "Personal observations on the reliability of the Shuttle," in Rogers Commission, *The Presidential Commission on the Space Shuttle Challenger Accident Report*. Washington, DC, June 1986. Also in <http://www.virtualschool.edu/mon./SocialConstruction/FeynmanChallengerRpt.html>.
- [IEEE Std. 829-1991] *IEEE Standard for Software Test Documentation*, IEEE Standards Board, Mar. 1991, in [IEEE 1997].
- [IEEE Std. 982-1989] *IEEE Guide for the Use of IEEE Standard Dictionary of Measures to Produce Reliable Software*, IEEE Standards Board, July 1989, in [IEEE 1997].
- [IEEE, 1997] *IEEE Standards Collection Software Engineering*, Piscataway, NJ, 1997.
- [Jones, 1977] T. C. Jones, "Programmer Quality and Programmer Productivity," IBM TR-02.764, 1977.
- [McCabe, 1976] T. McCabe, "A Software Complexity Measure," *IEEE Transactions on Software Engineering*, vol. 2, no. 12, Dec. 1976.
- [Myers, 1979] G. J. Myers, *The Art of Software Testing*, Wiley, New York, 1979.
- [Parnas & Weiss, 1985] D. L. Parnas & D. M. Weiss, "Active design reviews: principles and practice," *Proceedings of the Eight International Conference on Software Engineering*, pp 132–136, Aug. 1985.
- [Pfleeger, 1991] S. L. Pfleeger, *Software Engineering: The Production of Quality Software*, 2nd ed. Macmillan, 1991.
- [Popper, 1992] K. Popper, *Objective Knowledge: An Evolutionary Approach*. Clarendon, Oxford, 1992.
- [Siewiorek & Swarz, 1992] D. P. Siewiorek & R. S. Swarz, *Reliable Computer Systems: Design and Evaluation*. 2nd ed. Digital, Burlington, MA, 1992.
- [Turner & Robson, 1993] C. D. Turner & D. J. Robson, "The state-based testing of object-oriented programs," *Conference on Software Maintenance*. pp. 302–310, Sept. 1993.
- [Walker et al., 1980] B. J. Walker, R. A. Kemmerer, & G. J. Popek. "Specification and verification of the UCLA Unix security kernel," *Communications of the ACM* vol. 23, no. 2, pp. 118–131, 1980.

# 10

|             |                                                     |            |
|-------------|-----------------------------------------------------|------------|
| <b>10.1</b> | <b>Introduction: An aircraft example</b>            | <b>372</b> |
| <b>10.2</b> | <b>An overview of configuration management</b>      | <b>374</b> |
| <b>10.3</b> | <b>Configuration management concepts</b>            | <b>375</b> |
| 10.3.1      | Configuration items and CM aggregates               | 376        |
| 10.3.2      | Versions and configurations                         | 377        |
| 10.3.3      | Change requests                                     | 378        |
| 10.3.4      | Promotions and releases                             | 378        |
| 10.3.5      | Repositories and workspaces                         | 379        |
| 10.3.6      | Version identification schemes                      | 379        |
| 10.3.7      | Changes and change sets                             | 382        |
| 10.3.8      | Configuration management tools                      | 383        |
| <b>10.4</b> | <b>Configuration management activities</b>          | <b>384</b> |
| 10.4.1      | Configuration item and CM aggregate identification  | 387        |
| 10.4.2      | Promotion management                                | 389        |
| 10.4.3      | Release management                                  | 390        |
| 10.4.4      | Branch management                                   | 393        |
| 10.4.5      | Variant management                                  | 397        |
| 10.4.6      | Change management                                   | 400        |
| <b>10.5</b> | <b>Managing configuration management</b>            | <b>401</b> |
| 10.5.1      | Documenting configuration management                | 401        |
| 10.5.2      | Assigning configuration management responsibilities | 402        |
| 10.5.3      | Planning configuration management activities        | 403        |
| <b>10.6</b> | <b>Exercises</b>                                    | <b>403</b> |
|             | <b>References</b>                                   | <b>404</b> |